DATA-260 Homework 6 - Rachandeep Kaur

GitHub: https://github.com/rachandeepkaur/data260-7117

Section 0 — Personal Configuration and Domain

Value	Definition	My value
SID4	Last four digits of SJSU Student ID	7117
PORT_BASE	8000 + (SID4 mod 900)	8000 + 817 = 8817
PREFIX	"s" + SID4	s7117
SEED	SID4	7117
VERIFY_SEED	260000 + SID4	267117
DOMAIN_ID	SID4 mod 8	5 (local restaurant inspections)

Item	Value
Hardware	MacBook Pro — Intel Core i9-9880H @ 2.30 GHz (8 cores / 16 threads), 32 GB RAM, macOS 26.6.2; Ollama runs on the CPU
Local model	qwen3:8b via Ollama, called through the HW1 adapter src/model_client.py
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (fastembed, 384-d)
Database	MongoDB 7.0 (Docker container s7117-mongo), database s7117_doc
Backend	FastAPI on http://127.0.0.1:8817
Memory defaults	SHORT_TERM_N = 12, SUMMARIZE_EVERY_USER_MSGS = 5, lifetime summary every 2 session summaries, EPISODE_TOP_K = 3
Tagged commit	tag hw6

Part 1 — MongoDB Document API

The domain entity is one restaurant inspection document in s7117_doc.inspections. Fields: facility_name (required, 1–100 chars), description (optional), status (enum, default scheduled), program_element (category enum, 4 fixed options), inspection_date (required datetime), and server-set created_at / updated_at.

I. Database — collection and validation rules

The same rules are enforced twice: by Pydantic in the API and by a MongoDB $jsonSchema validator. A document inserted directly from mongosh is therefore also checked.

```python
def ensure_collections(db: Database | None = None) -> list[str]:
    """Create (or refresh the validator of) every HW6 collection; returns their names."""
    db = db if db is not None else get_db()
    for name, schema in VALIDATORS.items():
        validator = {"$jsonSchema": schema}
        try:
            db.create_collection(name, validator=validator, validationLevel="strict", validationAction="error")
        except CollectionInvalid:  # already exists
            db.command("collMod", name, validator=validator, validationLevel="strict", validationAction="error")
        for keys in INDEXES.get(name, []):
            db[name].create_index(keys)
    return list[str](VALIDATORS)
```

Figure 1. PyMongo connection and $jsonSchema validator (mongo.py).

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % docker exec s7117-mongo mongosh s7117_doc --quiet --eval 'printjson(db.getCollectionInfos({name: "inspections"})[0].options)'
{
  validator: {
    '$jsonSchema': {
      bsonType: 'object',
      required: [
        'facility_name',
        'status',
        'program_element',
        'inspection_date',
        'created_at',
        'updated_at'
      ],
      properties: {
        facility_name: {
          bsonType: 'string',
          minLength: 1,
          maxLength: 100
        },
        description: {
          bsonType: [
            'string',
            'null'
          ],
          maxLength: 2000
        },
        status: {
          enum: [
            'scheduled',
            'in_progress',
            'passed',
            'failed',
            'follow_up_required'
          ]
        },
        program_element: {
          enum: [
            'FOOD PREP / FOOD SVC OP 0-5 EMPLOYEES RC 3 - FP11',
            'FOOD PREP / FOOD SVC OP 6-25 EMPLOYEES',
            'ROUTINE HEALTH INSPECTION',
            'FOLLOW-UP COMPLIANCE CHECK'
          ]
        },
        inspection_date: {
          bsonType: 'date'
        },
        created_at: {
          bsonType: 'date'
        },
        updated_at: {
          bsonType: 'date'
        }
      }
    }
  },
  validationLevel: 'strict',
  validationAction: 'error'
}
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % docker exec s7117-mongo mongosh s7117_doc --quiet --eval 'db.inspections.insertOne({facility_name: "Direct insert", status: "bogus"})'
MongoServerError: Document failed validation
```

Figure 2. The validator stored on inspections, and MongoDB rejecting an invalid direct insert.

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % docker exec s7117-mongo mongosh s7117_doc --quiet --eval 'db.inspections.find().sort({created_at: -1}).limit(3)'
[
  {
    _id: ObjectId('6ad275f74a969ab5be8b8875'),
    facility_name: 'FA0206933 - YUMMY KITCHEN',
    description: 'Created by part1_doc_api_demo.py',
    status: 'scheduled',
    program_element: 'FOLLOW-UP COMPLIANCE CHECK',
    inspection_date: ISODate('2026-10-22T16:00:00.000Z'),
    created_at: ISODate('2026-10-16T19:07:35.181Z'),
    updated_at: ISODate('2026-10-16T19:07:35.181Z')
  },
  {
    _id: ObjectId('6ad1ec49f3bf08418bdb2430'),
    facility_name: 'FA0206933 - YUMMY KITCHEN',
    description: 'Hand-wash sink blocked by storage boxes; corrected on site.',
    status: 'scheduled',
    program_element: 'FOOD PREP / FOOD SVC OP 6-25 EMPLOYEES',
    inspection_date: ISODate('2026-09-01T16:00:00.000Z'),
    created_at: ISODate('2026-10-16T09:20:09.938Z'),
    updated_at: ISODate('2026-10-16T09:20:09.938Z')
  },
  {
    _id: ObjectId('6ad1ec49f3bf08418bdb2431'),
    facility_name: 'FA0148210 - GOLDEN DRAGON',
    description: 'No violations observed. Food handler cards current.',
    status: 'scheduled',
    program_element: 'FOLLOW-UP COMPLIANCE CHECK',
    inspection_date: ISODate('2026-09-04T16:00:00.000Z'),
    created_at: ISODate('2026-10-16T09:20:09.938Z'),
    updated_at: ISODate('2026-10-16T09:20:09.938Z')
  }
]
```

Figure 3. The inspections collection with sample documents.

II. API implementation

The code is split into server configuration (settings.py, app.py), the PyMongo connection (mongo.py), Pydantic schemas (doc_schemas.py) and route handlers (routers/doc_inspections.py). app.py adds CORS middleware and a JSON-body middleware: 415 if the body is not application/json, 400 if it does not parse. Error handlers return 422 for validation errors, 503 when MongoDB is down and 500 for anything else.

```python
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.getenv(
        "CORS_ORIGINS", "http://localhost:5173,http://127.0.0.1:5173"
    ).split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
    expose_headers=["X-Total-Count", "X-SQL-Query-Count", "X-Server-Time-Ms"],
)
```

```python
# before the route's Pydantic model validates it.
@app.middleware("http")
async def json_body_middleware(request: Request, call_next):
    if request.method in ("POST", "PUT", "PATCH") and request.url.path.startswith(HW6_PREFIXES):
        if "application/json" not in request.headers.get("content-type", ""):
            return JSONResponse(status_code=415, content={
                "error": "unsupported_media_type", "detail": "Send the request body as application/json"})
        body = await request.body()
        try:
            json.loads(body or b"null")
        except ValueError as exc:
            return JSONResponse(status_code=400, content={"error": "malformed_json", "detail": str(exc)})
    return await call_next(request)


@app.exception_handler(RequestValidationError)
async def validation_error_handler(request: Request, exc: RequestValidationError):
    if not request.url.path.startswith(HW6_PREFIXES):
        return await request_validation_exception_handler(request, exc)
    errors = [{"field": ".".join(str(p) for p in e["loc"] if p != "body") or "body",
               "message": e["msg"], "type": e["type"],
               **({"allowed": e["ctx"]["expected"]} if "expected" in e.get("ctx", {}) else {})}
              for e in exc.errors()]
    return JSONResponse(status_code=422, content={"error": "validation_error", "detail": errors})


@app.exception_handler(PyMongoError)
async def mongo_error_handler(_request: Request, exc: PyMongoError):
    return JSONResponse(status_code=503, content={"error": "database_unavailable",
                                                  "detail": f"MongoDB error: {type(exc).__name__}"})


@app.exception_handler(Exception)
async def server_error_handler(_request: Request, exc: Exception):
    return JSONResponse(status_code=500, content={"error": "internal_server_error", "detail": type(exc).__name__})
```

Figure 4. CORS middleware, JSON-parsing middleware and error handlers.

```python
class InspectorUpdate(BaseModel):
    full_name: Optional[str] = Field(default=None, min_length=2, max_length=120)
    district: Optional[str] = Field(default=None, min_length=2, max_length=120)
    email: Optional[EmailStr] = None

    @model_validator(mode="after")
    def _not_empty(self) -> "InspectorUpdate":
        if not self.model_fields_set:
            raise ValueError("Provide at least one field to update")
        return self


class InspectorOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    full_name: str
    district: str
    email: str
    created_at: datetime
    updated_at: datetime


def _get_or_404(db_session_basede26: Session, inspector_id: int) -> Inspector:
    inspector = db_session_basede26.get(Inspector, inspector_id)
    if inspector is None:
        raise HTTPException(status_code=404, detail=f"Inspector with id {inspector_id} not found")
    return inspector


def _commit(db_session_basede26: Session, inspector: Inspector) -> Inspector:
    try:
        db_session_basede26.commit()
    except IntegrityError as exc:
        db_session_basede26.rollback()
        raise HTTPException(status_code=409, detail=f"email {inspector.email} already exists") from exc
    db_session_basede26.refresh(inspector)
    return inspector


@router.post("", response_model=InspectorOut, status_code=201)
def create_inspector(
    payload: InspectorIn,
    _user: User = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db),
) -> Inspector:
    inspector = Inspector(
        full_name=payload.full_name.strip(),
        district=payload.district.strip(),
```

Figure 5. CRUD route handlers with 400/404/422 handling.

CRUD operations

Start the API with make run-api, then run these in a terminal (Postman is fine too: import reports/hw06/postman_collection.json).

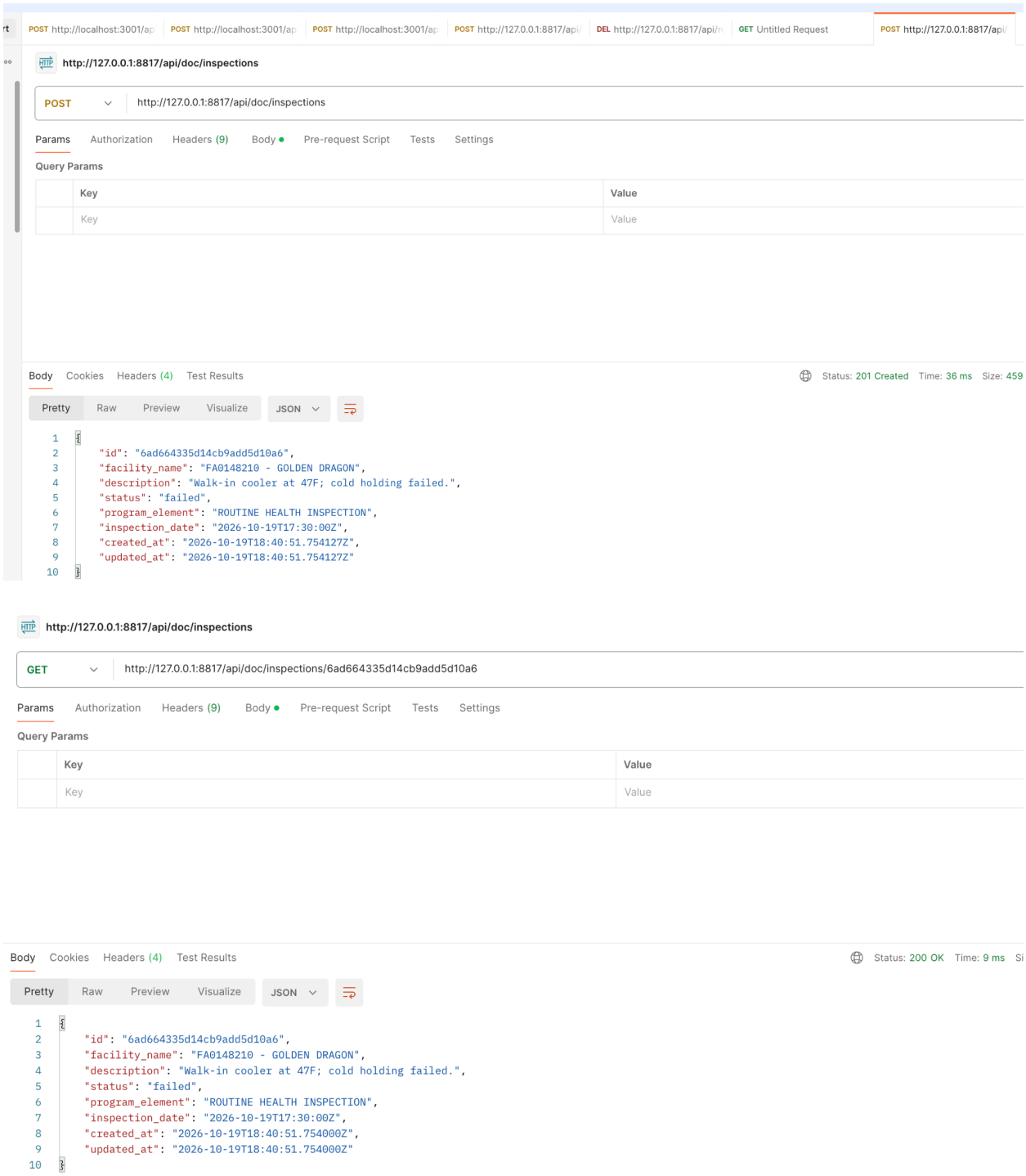


Figure 6. Create (POST → 201) and retrieve one by ID (GET /:id → 200).

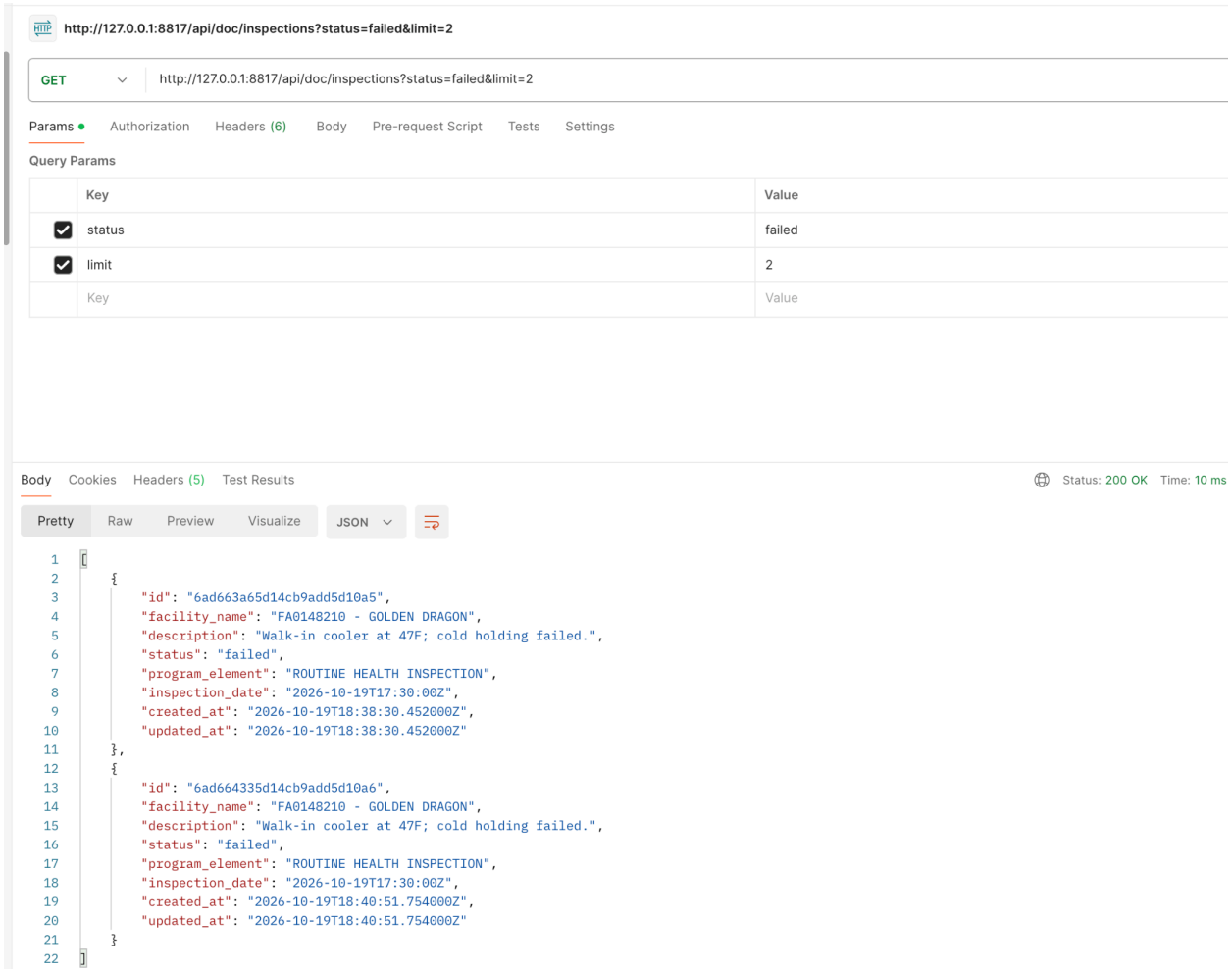


Figure 7. Retrieve all (GET → 200, with X-Total-Count) and filter by status.

http://127.0.0.1:8817/api/doc/inspections/<id>

PUT http://127.0.0.1:8817/api/doc/inspections/6ad664335d14cb9add5d10a6

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{"status": "follow_up_required", "description": "Follow-up moved to 2026-11-10."}
```

Body Cookies Headers (4) Test Results Status: 200 OK Time: 9

Pretty Raw Preview Visualize JSON


```
{
    "id": "6ad664335d14cb9add5d10a6",
    "facility_name": "FA0148210 - GOLDEN DRAGON",
    "description": "Follow-up moved to 2026-11-10.",
    "status": "follow_up_required",
    "program_element": "ROUTINE HEALTH INSPECTION",
    "inspection_date": "2026-10-19T17:30:00Z",
    "created_at": "2026-10-19T18:40:51.754000Z",
    "updated_at": "2026-10-19T18:45:17.354000Z"
}
```


http://127.0.0.1:8817/api/doc/inspections/<id>

DELETE http://127.0.0.1:8817/api/doc/inspections/6ad664335d14cb9add5d10a6

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Content-Type	~~text/plain~~
Content-Length	<calculated when request is sent>
Host	<calculated when request is sent>
User-Agent	PostmanRuntime/7.56.1
Accept	*/*
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Content-Type	application/json
Key	Value

Body Cookies Headers (4) Test Results Status: 200 OK Time: 8 ms Size

Pretty Raw Preview Visualize JSON


```
{
    "deleted": true,
    "id": "6ad664335d14cb9add5d10a6"
}
```

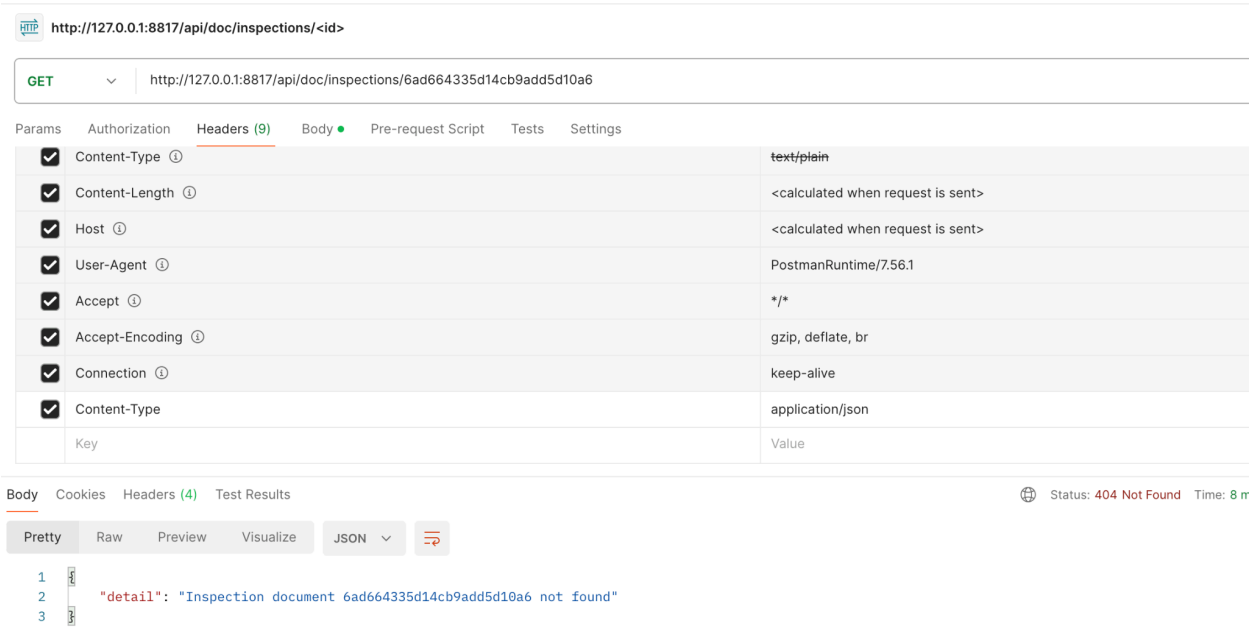


Figure 8. Update (PUT → 200, updated_at moves), delete (DELETE → 200), then GET → 404.

III. Data validation

```python
class InspectionStatus(str, Enum):
    scheduled = "scheduled"
    in_progress = "in_progress"
    passed = "passed"
    failed = "failed"
    follow_up_required = "follow_up_required"


class ProgramElement(str, Enum):
    fp11 = "FOOD PREP / FOOD SVC OP 0-5 EMPLOYEES RC 3 - FP11"
    fp_6_25 = "FOOD PREP / FOOD SVC OP 6-25 EMPLOYEES"
    routine = "ROUTINE HEALTH INSPECTION"
    follow_up = "FOLLOW-UP COMPLIANCE CHECK"


STATUSES = tuple[str, ...](s.value for s in InspectionStatus)
CATEGORIES = tuple[str, ...](c.value for c in ProgramElement)


def _strip_required(value: str) -> str:
    value = value.strip()
    if not value:
        raise ValueError("must not be blank")
    return value


class InspectionDocCreate(BaseModel):
    # extra="forbid" so a typo such as "staus" is reported instead of ignored,
    # and clients cannot set created_at/updated_at themselves.
    model_config = ConfigDict(extra="forbid", use_enum_values=True)

    facility_name: str = Field(min_length=1, max_length=100, examples=["FA0206933 - YUMMY KITCHEN"])
    description: Optional[str] = Field(default=None, max_length=2000)
    status: InspectionStatus = InspectionStatus.scheduled
    program_element: ProgramElement
    inspection_date: datetime = Field(examples=["2026-10-20T09:30:00Z"])

    _strip = field_validator("facility_name")(_strip_required)


class InspectionDocUpdate(BaseModel):
    """PUT body: every field optional, but at least one must be sent."""
    model_config = ConfigDict(extra="forbid", use_enum_values=True)

    facility_name: Optional[str] = Field(default=None, min_length=1, max_length=100)
    description: Optional[str] = Field(default=None, max_length=2000)
    status: Optional[InspectionStatus] = None
    program_element: Optional[ProgramElement] = None
    inspection_date: Optional[datetime] = None

    @field_validator("facility_name")
    @classmethod
    def _strip(cls, value: Optional[str]) -> Optional[str]:
        return None if value is None else _strip_required(value)
```

Figure 9. Pydantic schemas: enums, string lengths, required datetime, extra="forbid", non-empty update.

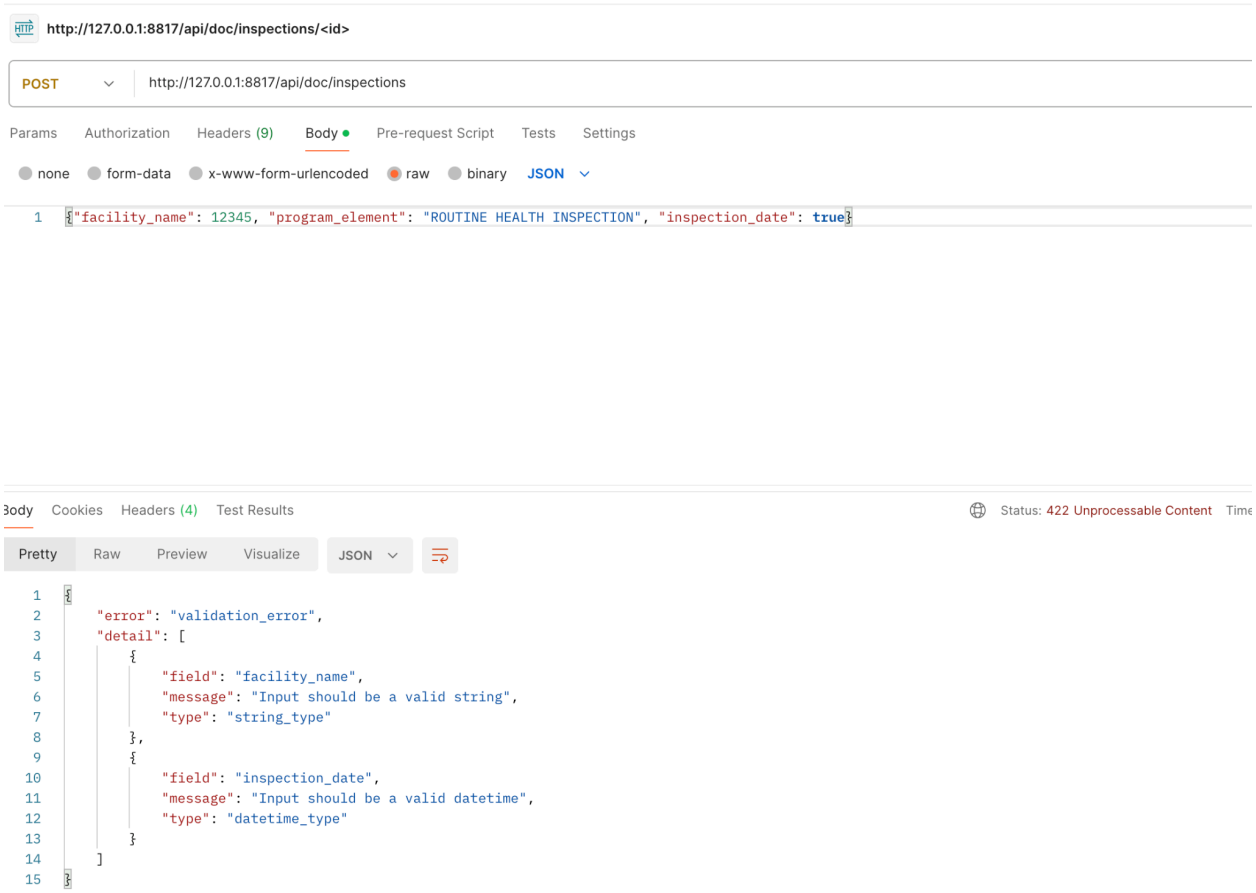


Figure 10. Invalid data → 422 with field-level messages.

make doc-api-demo-hw06 runs the same CRUD and 15 error cases: **21/21** calls returned the expected status (raw/part1_doc_api_calls.json).

Part 2 — Short-, Long-, and Episodic Memory

The same s7117_doc database gets three new collections, each with its own $jsonSchema validator: messages, summaries (session_id = null for the lifetime summary, scope ∈ {session, user}) and episodes (fact, importance 0..1, 384-float embedding). The Part 1 collection is unchanged.

MongoDB collections

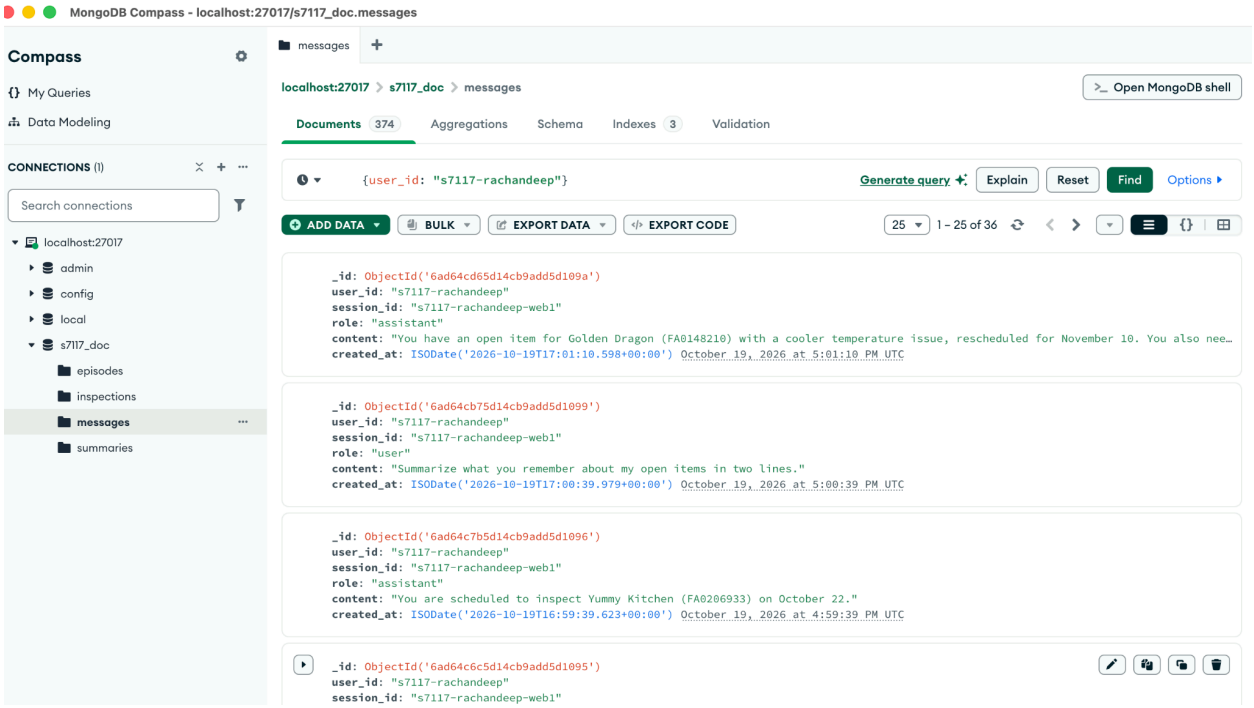


Figure 12. messages collection.

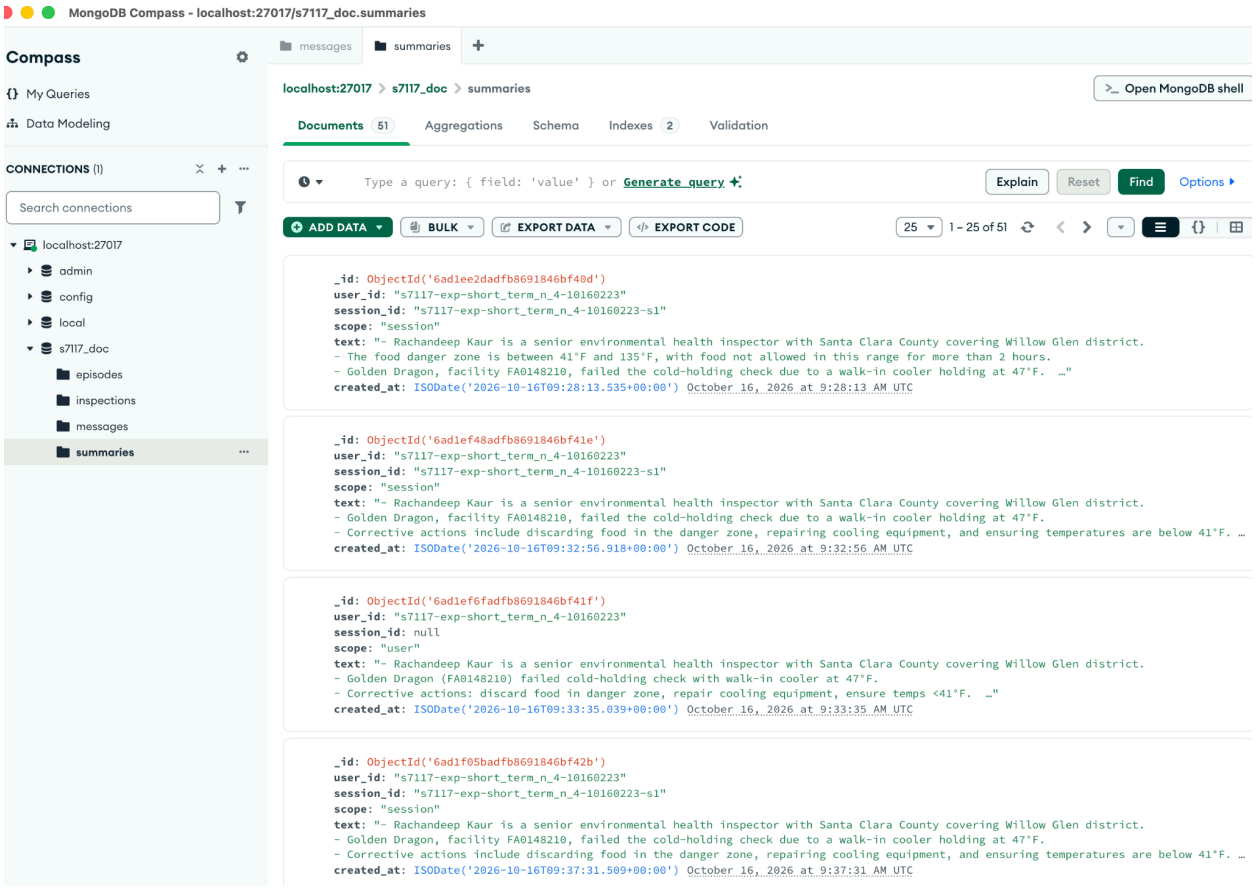


Figure 13. summaries — session summaries and the lifetime summary (scope 'user', session_id null).

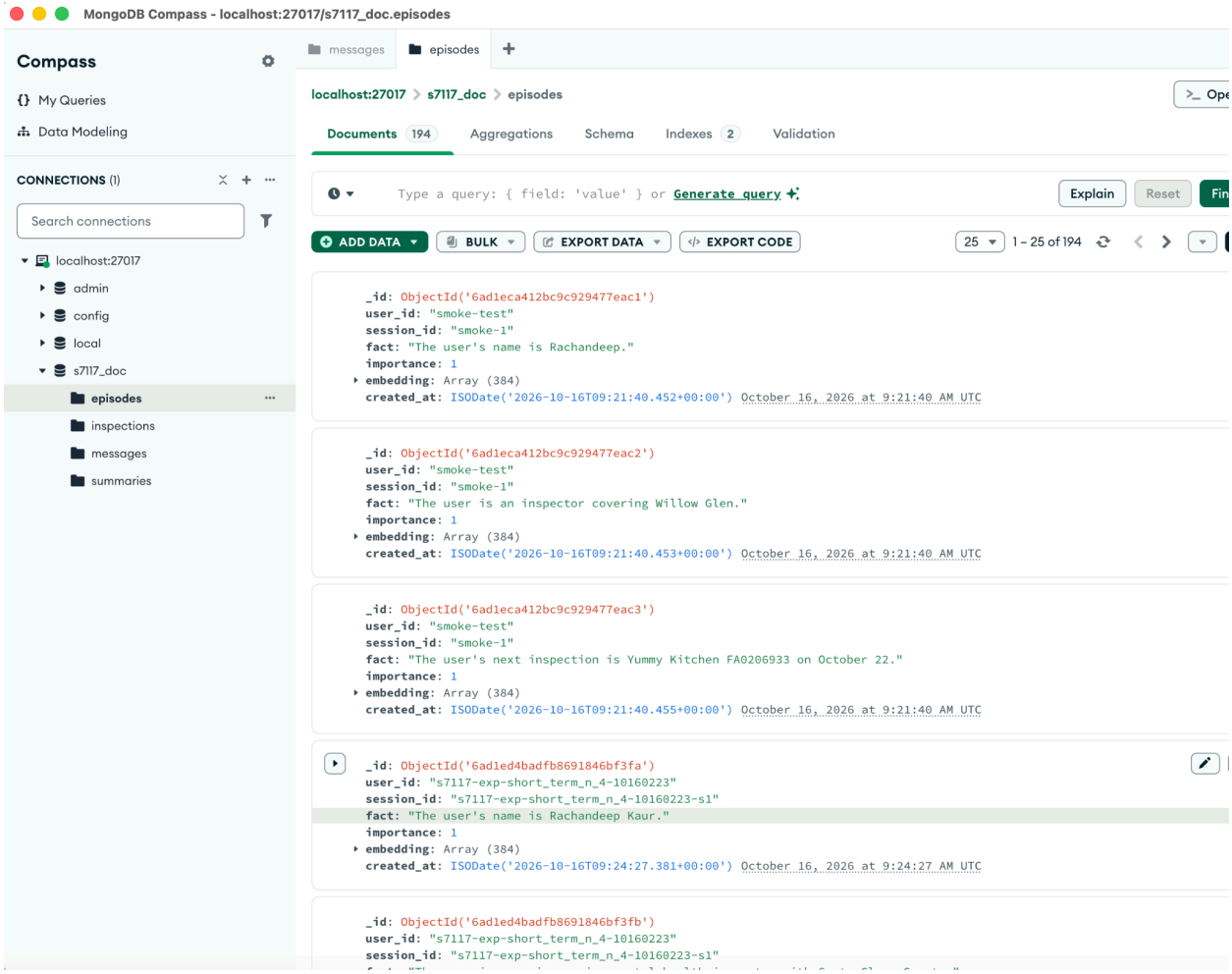


Figure 14. episodes with importance and embedding.

Endpoints

```python
@router.post("/api/chat", response_model=ChatResponse)
def chat(req: ChatRequest) -> dict:
    try:
        return memory_service.handle_chat(get_db(), req)
    except memory_service.ChatModelError as exc:
        raise HTTPException(status_code=502, detail=f"Local model call failed: {exc}") from exc


@router.get("/api/memory/{user_id}")
def get_memory(user_id: str, messages: int = Query(default=16, ge=1, le=200),
               facts: int = Query(default=20, ge=1, le=200)) -> dict:
    db = get_db()
    recent = list[Any](db[settings.MESSAGES].find({"user_id": user_id})
                .sort([("created_at", DESCENDING), ("_id", DESCENDING)]).limit(messages))
    if not recent and db[settings.EPISODES].count_documents({"user_id": user_id}, limit=1) == 0:
        raise HTTPException(status_code=404, detail=f"No memory stored for user '{user_id}'")
    session = db[settings.SUMMARIES].find_one({"user_id": user_id, "scope": "session"},
                                              sort=[("created_at", DESCENDING)])
    lifetime = db[settings.SUMMARIES].find_one({"user_id": user_id, "scope": "user"},
                                               sort=[("created_at", DESCENDING)])
    episodes = db[settings.EPISODES].find({"user_id": user_id}, {"embedding": 0}) \
        .sort("created_at", DESCENDING).limit(facts)
    dims = db[settings.EPISODES].find_one({"user_id": user_id}, {"embedding": 1})
    return {
        "user_id": user_id,
        "messages": [_iso(m) for m in reversed[Any](recent)],
        "session_summary": _iso(session) if session else None,
        "lifetime_summary": _iso(lifetime) if lifetime else None,
        "episodic_facts": [_iso(e) for e in episodes],
        "embedding_dim": len(dims["embedding"]) if dims else None,
    }


@router.get("/api/aggregate/{user_id}")
def aggregate(user_id: str, tz: str = Query(default="America/Los_Angeles"),
              summaries: int = Query(default=5, ge=1, le=50)) -> dict:
    db = get_db()
    daily = list[Any](db[settings.MESSAGES].aggregate([
        {"$match": {"user_id": user_id}},
        {"$group": {
            "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at", "timezone": tz}},
            "messages": {"$sum": 1},
            "user_messages": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}},
            "assistant_messages": {"$sum": {"$cond": [{"$eq": ["$role", "assistant"]}, 1, 0]}},
            "sessions": {"$addToSet": "$session_id"},
        }},
```

Figure 15. The three endpoints.

Memory logic

```python
def build_prompt(window: list[dict], message: str, session_summary: Optional[str],
                 lifetime_summary: Optional[str], facts: list[dict]) -> tuple[list[dict], dic
    """Prompt assembly: system primer + long-term summaries | short-term window | current mes

    Returns the chat messages and the text of each component (for token accounting).
    """
    long_term = ""
    if lifetime_summary:
        long_term += f"\n\n## Lifetime summary (all sessions)\n{lifetime_summary}"
    if session_summary:
        long_term += f"\n\n## Summary of this session so far\n{session_summary}"
    episodic = ""
    if facts:
        lines = "\n".join(f"- [{f['created_at'][:16].replace('T', ' ')}] {f['fact']}" for f i
        episodic = f"\n\n[Remembered facts relevant to this message, newest date wins]\n{line
    messages = [{"role": "system", "content": SYSTEM_PRIMER + long_term}, *window,
                {"role": "user", "content": message + episodic}]
    return messages, {"system_primer": SYSTEM_PRIMER, "long_term": long_term, "episodic": epi
```

```python
def handle_chat(db: Database, req: ChatRequest) -> dict:
    t0 = time.perf_counter()
    session_id = req.session_id or f"{req.user_id}-{uuid.uuid4().hex[:8]}"
    aux_usage: list[dict] = []

    user_doc = save_message(db, req.user_id, session_id, "user", req.message)
    window = load_short_term(db, req.user_id, session_id, settings.SHORT_TERM_N, before_id=us
    session_summary = latest_summary(db, req.user_id, "session", session_id)
    lifetime_summary = latest_summary(db, req.user_id, "user")
    query_vec = embed([req.message])[0]
    facts = retrieve_episodes(db, req.user_id, query_vec)
    window, facts, policy_info = apply_memory_policy(window, facts)

    messages, parts = build_prompt(window, req.message, session_summary and session_summary["
                                   lifetime_summary and lifetime_summary["text"], facts)
    client = ModelClient()
    t_model = time.perf_counter()
    try:
        reply = _clean(client.complete(messages, max_tokens=settings.CHAT_MAX_TOKENS))
    except ModelClientError as exc:
        db[settings.MESSAGES].delete_one({"_id": user_doc["_id"]})  # keep history consistent
        raise ChatModelError(str(exc)) from exc
    model_latency_ms = (time.perf_counter() - t_model) * 1000
    usage_main = dict[str, int](client.last_usage)
    save_message(db, req.user_id, session_id, "assistant", reply)

    extracted = store_episodes(db, req.user_id, session_id, extract_facts(req.message, aux_us
    updates = maybe_summarize(db, req.user_id, session_id, aux_usage)
    latency_ms = (time.perf_counter() - t0) * 1000
```

Figure 16. Prompt assembly and the /api/chat server steps in order.

```python
def extract_facts(message: str, aux_usage: list) -> list[dict]:
    """Episode extraction: up to MAX_FACTS_PER_MESSAGE short facts with an importance score."""
    client = ModelClient()
    t0 = time.perf_counter()
    try:
        result = client.call(system=EXTRACT_SYSTEM, user=f"User message:\n{message}", output_format=ExtractedF
                             max_tokens=220, max_attempts=2)
        facts = [f.model_dump() for f in result.facts[: settings.MAX_FACTS_PER_MESSAGE]]
    except ModelClientError:
        facts = []
    aux_usage.append({**client.last_usage, "latency_ms": round((time.perf_counter() - t0) * 1000, 1)})
    return facts


def store_episodes(db: Database, user_id: str, session_id: str, facts: list[dict]) -> list[dict]:
    """Embed each fact and store it; a near-duplicate (cos >= 0.92) refreshes the stored one instead."""
    if not facts:
        return []
    vectors = embed([f["fact"] for f in facts])
    existing = list[Any](db[settings.EPISODES].find({"user_id": user_id}, {"embedding": 1, "importance": 1}))
    matrix = np.asarray([e["embedding"] for e in existing]) if existing else None
    stored = []
    for fact, vec in zip[tuple[dict, list[float]]](facts, vectors):
        importance = float(round(fact["importance"], 3))
        if matrix is not None:
            sims = cosine(np.asarray(vec), matrix)
            best = int(np.argmax(sims))
            if sims[best] >= settings.EPISODE_DEDUP_SIMILARITY:
                db[settings.EPISODES].update_one({"_id": existing[best]["_id"]}, {"$set": {
                    "fact": fact["fact"], "embedding": vec, "session_id": session_id, "created_at": _now(),
                    "importance": max(importance, float(existing[best]["importance"]))}})
                stored.append({"fact": fact["fact"], "importance": importance, "action": "merged"})
                continue
        db[settings.EPISODES].insert_one({"user_id": user_id, "session_id": session_id, "fact": fact["fact"],
                                          "importance": importance, "embedding": vec, "created_at": _now()})
        stored.append({"fact": fact["fact"], "importance": importance, "action": "inserted"})
    return stored
```

Figure 17. Episode extraction (≤ 3 facts with importance) and storing each fact's embedding; near-duplicates refresh the stored fact.



Figure 18. Session summary every M = 5 user messages; lifetime summary (scope 'user', session_id null) every 2 session summaries.

```python
def embed(texts: list[str]) -> list[list[float]]:
    return [[float(x) for x in v] for v in _embedder().embed(texts)]


def cosine(query: np.ndarray, matrix: np.ndarray) -> np.ndarray:
    q = query / (np.linalg.norm(query) or 1.0)
    m = matrix / np.clip(np.linalg.norm(matrix, axis=1, keepdims=True), 1e-12, None)
    return m @ q


def retrieve_episodes(db: Database, user_id: str, query_vec: list[float], k: int = settings.EPISODE_TOP_K,
                      min_similarity: float = settings.EPISODE_MIN_SIMILARITY) -> list[dict]:
    """Top-k episodes of this user by cosine similarity to the current message."""
    episodes = list[Any](db[settings.EPISODES].find({"user_id": user_id},
                                                    {"fact": 1, "importance": 1, "embedding": 1, "created_at": 1}))
    if not episodes or k <= 0:
        return []
    scores = cosine(np.asarray(query_vec), np.asarray([e["embedding"] for e in episodes]))
    ranked = sorted(zip[tuple[Any, Any]](episodes, scores), key=lambda p: p[1], reverse=True)
    return [{"id": str(e["_id"]), "fact": e["fact"], "importance": e["importance"], "similarity": round(float(s),
             "created_at": e["created_at"].isoformat()}
            for e, s in ranked[:k] if s >= min_similarity]
```

Figure 19. Embedding and top-k retrieval by cosine similarity.

Run and test

make chat-demo-hw06 sent **12 messages in one session** (s7117-rachandeep-demo1). The window grows 0→12 messages and then slides. A session summary is written at user messages 5 and 10, the lifetime summary at message 10, and facts are retrieved from turn 3 on. Turn 11 recalls the rescheduled follow-up (Nov 3 → Nov 10) and turn 12 the supervisor.

```
===== [2026-10-16 12:07:35 PDT] make chat-demo-hw06 (user: rachandeep Kaur) =====
HW6 Part 2 - chat memory demo - user: rachandeep Kaur - 2026-10-16 12:07:35
user_id=s7117-rachandeep session_id=s7117-rachandeep-demo1

[ 1] rachandeep Kaur: Hi, I'm Rachandeep Kaur, a Santa Clara County food inspector. I cover the Willow Glen district.
     InspectBot: Hello, Rachandeep Kaur. I'm InspectBot for the s7117 restaurant-inspection app. I'll assist with your inspections in the Willow Glen district.
     memory used: short_term=0 msgs, session_summary=no, lifetime_summary=no, facts=[]
     updates: extracted=["The user's name is Rachandeep Kaur.", 'The user is a Santa Clara County food inspector.', 'The user covers the Willow Glen district.'
     tokens prompt=135 completion=46 latency=33406ms

[ 2] rachandeep Kaur: Today Golden Dragon (FA0148210) failed cold holding: the walk-in cooler was at 47F.
     InspectBot: Golden Dragon (FA0148210) failed cold holding with the walk-in cooler at 47°F. This temperature is above the acceptable range. The issue needs
     memory used: short_term=2 msgs, session_summary=no, lifetime_summary=no, facts=[]
     updates: extracted=['The user mentioned Golden Dragon (FA0148210) failed cold holding.', 'The walk-in cooler was at 47F.'] session_summary_updated=False l
     tokens prompt=218 completion=50 latency=27281ms

[ 3] rachandeep Kaur: I set their follow-up inspection for November 3.
     InspectBot: Follow-up inspection for Golden Dragon (FA0148210) is scheduled for November 3. I've noted the date in the system. Let me know if you need fur
     memory used: short_term=4 msgs, session_summary=no, lifetime_summary=no, facts=['The user is a Santa Clara County food inspector.']
     updates: extracted=['The user scheduled a follow-up inspection for November 3.'] session_summary_updated=False lifetime_summary_updated=False
     tokens prompt=330 completion=42 latency=20722ms

[ 4] rachandeep Kaur: What should they do right away about the cooler?
     InspectBot: The cooler temperature was 47°F, which is too high. They should immediately repair or adjust the cooler to ensure it reaches the proper temper
     memory used: short_term=6 msgs, session_summary=no, lifetime_summary=no, facts=['The walk-in cooler was at 47F.']
     updates: extracted=[] session_summary_updated=False lifetime_summary_updated=False
     tokens prompt=392 completion=43 latency=15038ms
```

```
[10] rachandeep Kaur: Draft a two-line note to my supervisor about Golden Dragon.
     InspectBot: The follow-up inspection for Golden Dragon (FA0148210) has been rescheduled to November 10 due to a delayed part. The cold-holding failure
     memory used: short_term=12 msgs, session_summary=yes, lifetime_summary=no, facts=['The Golden Dragon follow-up was moved to November 10.', 'The user m
     updates: extracted=[] session_summary_updated=True lifetime_summary_updated=True
     tokens prompt=759 completion=46 latency=143610ms

[11] rachandeep Kaur: Which facility had the cooler problem, and when is its follow-up?
     InspectBot: The facility with the cooler problem is Golden Dragon (FA0148210). Its follow-up inspection is scheduled for November 10.
     memory used: short_term=12 msgs, session_summary=yes, lifetime_summary=yes, facts=['The walk-in cooler was at 47F.', 'The user is asking about the saf
     updates: extracted=['The facility had a cooler problem.', "The facility's follow-up is scheduled for a specific date."] session_summary_updated=False
     tokens prompt=940 completion=32 latency=55627ms

[12] rachandeep Kaur: What do you remember about me?
     InspectBot: You are preparing for inspections in the San Jose area, focusing on Golden Dragon (FA0148210) and Yummy Kitchen (FA0206933). You need to b
     memory used: short_term=12 msgs, session_summary=yes, lifetime_summary=yes, facts=[]
     updates: extracted=[] session_summary_updated=False lifetime_summary_updated=False
     tokens prompt=815 completion=69 latency=35435ms
```

Figure 20. Console log of the 12-message session with the memory used and updated each turn.

A second session (s7117-rachandeep-web1) was then run in the mini chat page. On its first turn the window is empty, yet the reply already uses Golden Dragon from the earlier session through episodic facts and the lifetime summary.

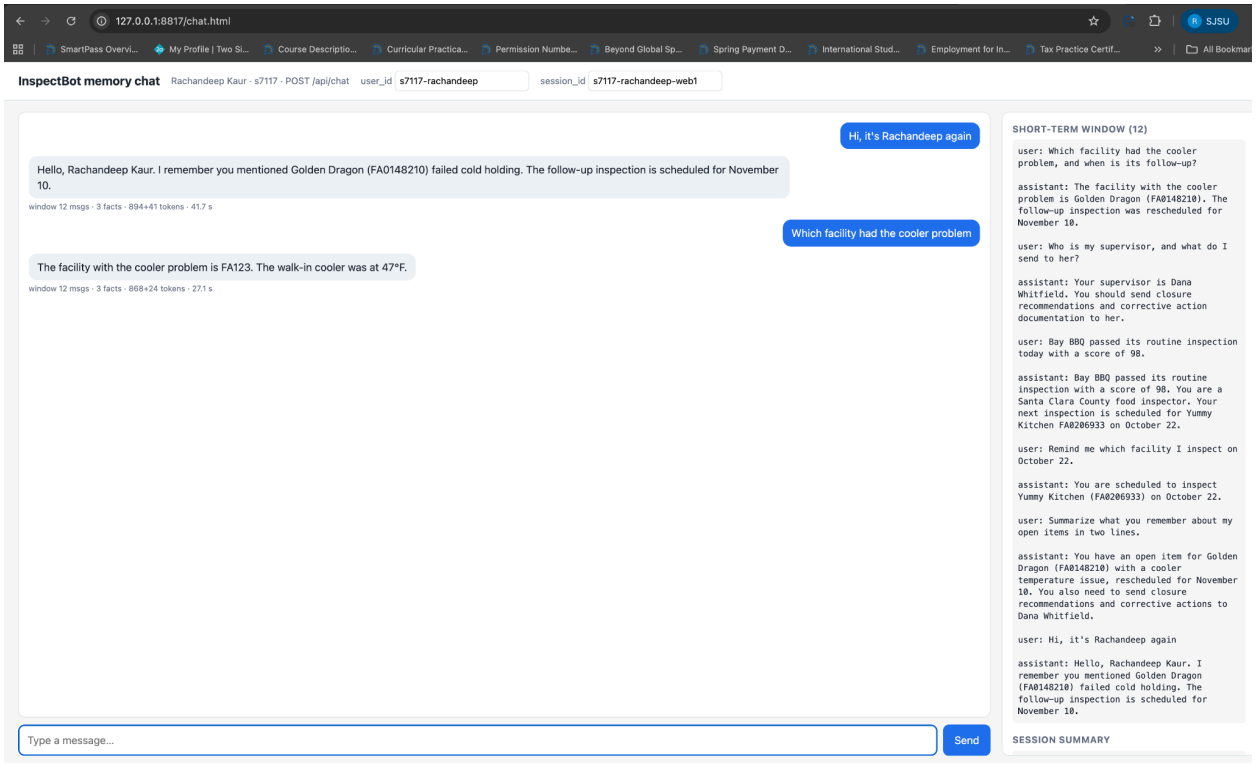


Figure 21. Mini chat page (POST /api/chat): assistant replies, with the memory used for the turn on the right.

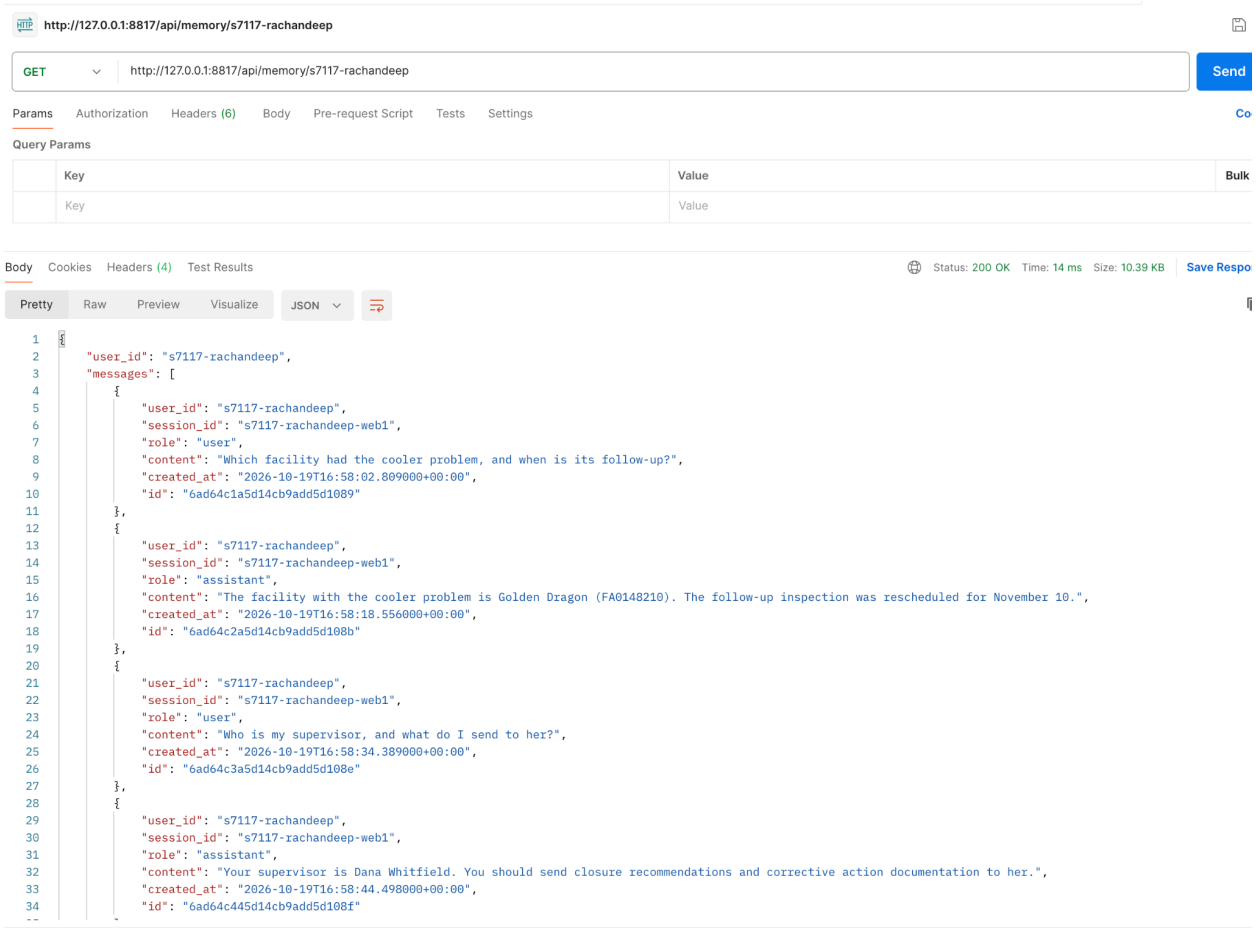


Figure 22. GET /api/memory: last 16 messages, latest session and lifetime summaries, last 20 episodic facts.

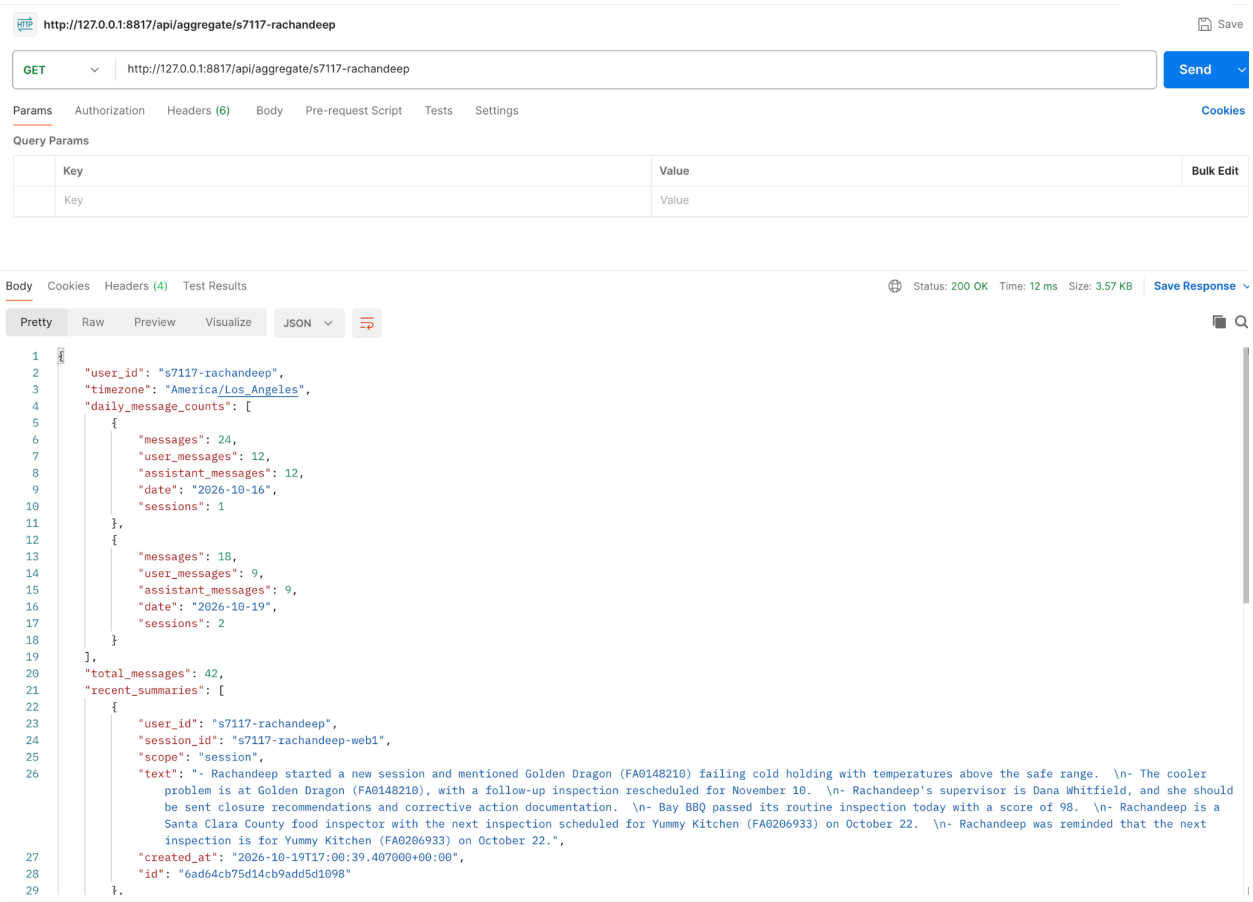


Figure 23. GET /api/aggregate: daily message counts and the most recent summaries.

Part 3 — What Memory Costs

SHORT_TERM_N=4 is the best setting: $287.05 per 10,000 conversations against $548.00 for N=30, for a 0.4-point drop in probe quality.

1. Logging memory usage

Every /api/chat response carries a usage object: prompt and completion tokens as reported by Ollama, per-component token counts (Qwen3 tokenizer) for the system primer, long-term summary, short-term window and episodic facts, and the latencies. The experiment runner writes one row per request to raw/turns_*.jsonl and raw/memory_turns.csv.

```
    tc = token_counter
    usage = {
        "prompt_tokens": usage_main.get("input_tokens", 0),          # reported by Ollama
        "completion_tokens": usage_main.get("output_tokens", 0),
        "components": {
            "system_primer": tc.count_tokens(parts["system_primer"]) + tc.CHATML_OVERHEAD_PER_MESSAGE,
            "long_term_summary": tc.count_tokens(parts["long_term"]),
            "short_term_window": tc.count_messages(window),
            "episodic_facts": tc.count_tokens(parts["episodic"]),
            "current_message": tc.count_tokens(req.message) + tc.CHATML_OVERHEAD_PER_MESSAGE,
        },
        "aux_prompt_tokens": sum(u.get("input_tokens", 0) for u in aux_usage),
        "aux_completion_tokens": sum(u.get("output_tokens", 0) for u in aux_usage),
        "aux_calls": len(aux_usage),
        "model_latency_ms": round(model_latency_ms, 1),
        "aux_latency_ms": round(sum(u["latency_ms"] for u in aux_usage), 1),
        "latency_ms": round(latency_ms, 1),
        "tokenizer": tc.tokenizer_name(),
    }
    usage["components"]["counted_total"] = sum(usage["components"].values())
```

```
    if settings.USAGE_LOG:
        row = {"ts": _now().isoformat(), "user_id": req.user_id, "session_id": session_id,
               "config": result["config"], **usage}
        with open(settings.USAGE_LOG, "a") as fh:
            fh.write(json.dumps(row) + "\n")
    return result
```

Figure 24. Per-request usage record (handle_chat).

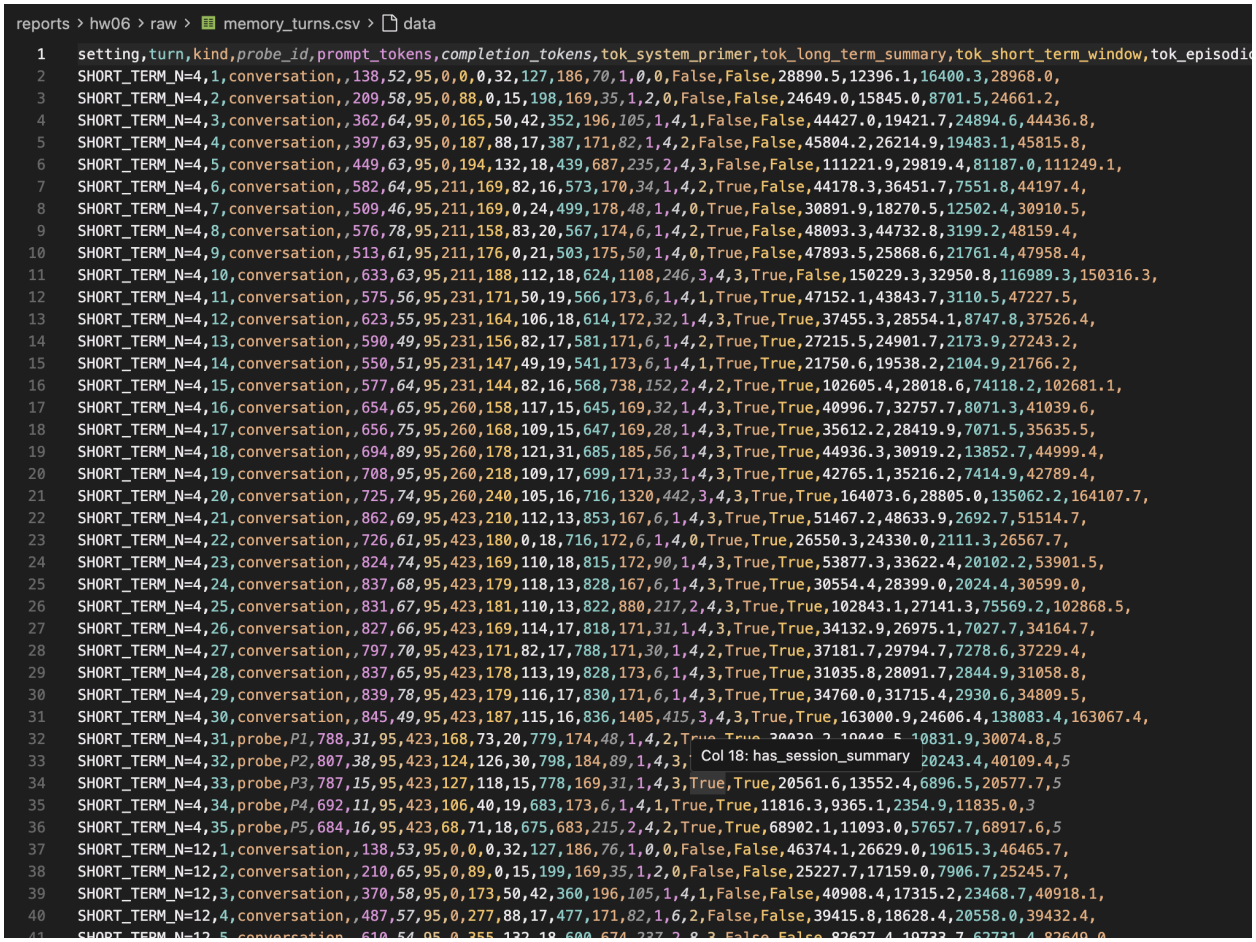
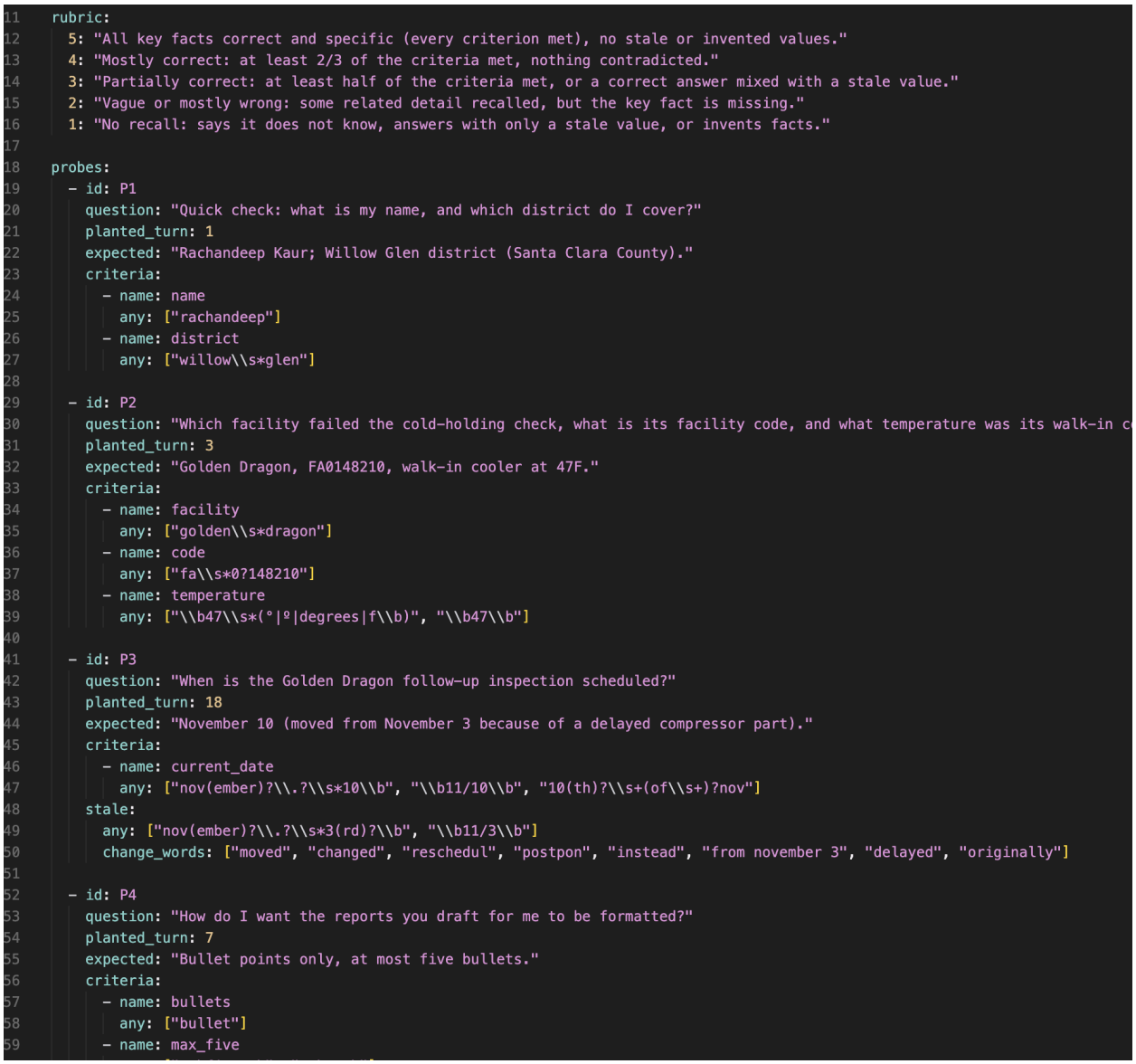


Figure 25. Raw files and a sample of memory_turns.csv.

2. Memory probes

memory_probes.yaml was written before any run and kept fixed. It holds five questions about facts planted at turns 1, 3, 7, 9 and 18, regex criteria per probe, a stale-value penalty, and a 1–5 rubric. Scoring is automatic (probe_scoring.py).



```yaml
rubric:
  5: "All key facts correct and specific (every criterion met), no stale or invented values."
  4: "Mostly correct: at least 2/3 of the criteria met, nothing contradicted."
  3: "Partially correct: at least half of the criteria met, or a correct answer mixed with a stale value."
  2: "Vague or mostly wrong: some related detail recalled, but the key fact is missing."
  1: "No recall: says it does not know, answers with only a stale value, or invents facts."

probes:
  - id: P1
    question: "Quick check: what is my name, and which district do I cover?"
    planted_turn: 1
    expected: "Rachandeep Kaur; Willow Glen district (Santa Clara County)."
    criteria:
      - name: name
        any: ["rachandeep"]
      - name: district
        any: ["willow\\s*glen"]

  - id: P2
    question: "Which facility failed the cold-holding check, what is its facility code, and what temperature was its walk-in c
    planted_turn: 3
    expected: "Golden Dragon, FA0148210, walk-in cooler at 47F."
    criteria:
      - name: facility
        any: ["golden\\s*dragon"]
      - name: code
        any: ["fa\\s*0?148210"]
      - name: temperature
        any: ["\\b47\\s*(°|º|degrees|f\\b)", "\\b47\\b"]

  - id: P3
    question: "When is the Golden Dragon follow-up inspection scheduled?"
    planted_turn: 18
    expected: "November 10 (moved from November 3 because of a delayed compressor part)."
    criteria:
      - name: current_date
        any: ["nov(ember)?\\.?\\s*10\\b", "\\b11/10\\b", "10(th)?\\s+(of\\s+)?nov"]
    stale:
      any: ["nov(ember)?\\.?\\s*3(rd)?\\b", "\\b11/3\\b"]
      change_words: ["moved", "changed", "reschedul", "postpon", "instead", "from november 3", "delayed", "originally"]

  - id: P4
    question: "How do I want the reports you draft for me to be formatted?"
    planted_turn: 7
    expected: "Bullet points only, at most five bullets."
    criteria:
      - name: bullets
        any: ["bullet"]
      - name: max_five
```

Figure 26. memory_probes.yaml: rubric, questions and expected-answer criteria.

3. Short-term memory sizes

The same 30-message conversation (conversation_30.yaml) was run in a fresh session per setting, then the 5 probes were asked. The runner restarts the API with each setting's environment variables.

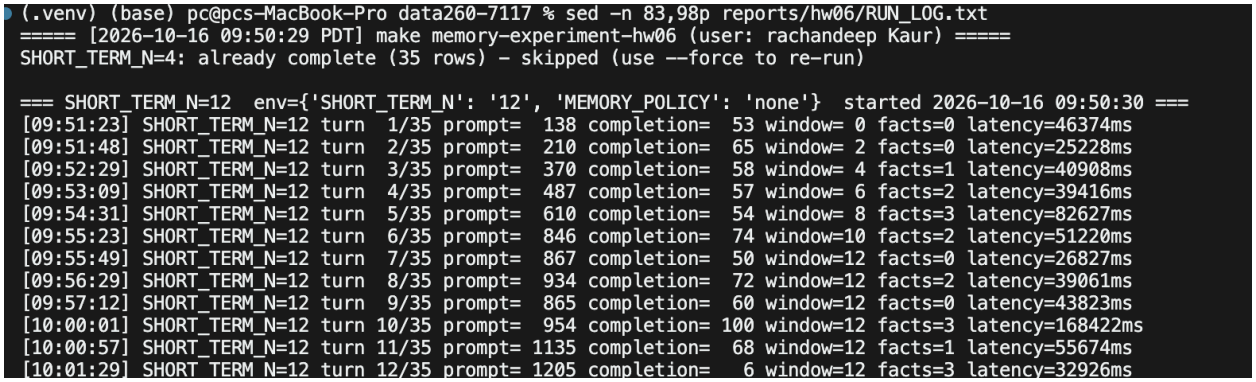


Figure 27. Per-turn tokens, window size, facts and latency during the experiment.

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	631.5	56,875	4.6
SHORT_TERM_N=12	964.8	61,394	4.6
SHORT_TERM_N=30	1,489.0	81,569	5.0
Eviction/compression policy (N=12)	576.7	43,548	3.8

Setting	Primer	Long-term summary	Short-term window	Episodic facts	Completion	P1	P2	P3	P4	P5
N=4	95.0	258.0	168.0	82.2	65.1	5	5	5	3	5
N=12	95.0	279.2	480.2	82.2	67.9	5	5	5	3	5
N=30	95.0	265.0	1,018.5	82.2	67.5	5	5	5	5	5
Compression	95.0	177.7	255.5	19.8	46.9	3	5	5	3	3

Mean tokens per turn by prompt component, and the probe score for each question.

Only the short-term window grows with N (168 → 1,019 tokens). The primer, summaries and facts stay about the same, so N=30 sends 2.4× the prompt tokens of N=4 and is 43% slower. Quality barely moves because the summaries and facts carry the old facts. The one difference is P4 (report format, planted at turn 7): only N=30 still had that turn in its window, and the summaries kept "bullet points" but dropped "at most five". Latency includes about 20–28 s per turn of memory upkeep (fact extraction and summaries) on top of the chat call.

4. Memory-saving policy: compression + eviction

With MEMORY_POLICY=compress, the newest 2 messages stay verbatim. Older assistant turns in the window are cut to their first sentence (≤ 160 chars), user turns are never shortened, and episodic facts with cosine < 0.40 are evicted.

```python
def apply_memory_policy(window: list[dict], facts: list[dict], policy: str = settings.MEMORY_POLICY):
    """Part 3.4 compression policy.

    "compress": the newest COMPRESS_KEEP_VERBATIM messages stay verbatim; older
    *assistant* turns in the window are cut to their first sentence (<=160
    chars) because they mostly restate what the user said, while user turns -
    where the facts come from - are never shortened; episodic facts below
    cosine 0.40 are evicted as weak matches.
    """
    if policy != "compress":
        return window, facts, {"policy": "none"}
    keep = settings.COMPRESS_KEEP_VERBATIM
    out, truncated = [], 0
    for i, m in enumerate[dict](window):
        if m["role"] == "assistant" and i < len(window) - keep:
            short = _first_sentence(m["content"], settings.COMPRESS_ASSISTANT_CHARS)
            truncated += short != m["content"]
            out.append({"role": "assistant", "content": short})
        else:
            out.append(m)
    kept = [f for f in facts if f["similarity"] >= settings.COMPRESS_MIN_SIMILARITY]
    return out, kept, {"policy": "compress", "assistant_turns_truncated": truncated,
                       "facts_evicted": len(facts) - len(kept)}
```

Figure 28. The compression/eviction policy.

Metric	SHORT_TERM_N=12	Compression (N=12)	Change
Mean prompt tokens/turn	964.8	576.7	−388.1 (−40.2%)
Short-term window tokens/turn	480.2	255.5	−224.7
Episodic-fact tokens/turn	82.2	19.8	−62.4
Input tokens per 30-turn conversation	28,945	17,301	−11,644
Mean latency (ms)	61,394	43,548	−17,846
Probe quality (1-5)	4.6	3.8	−0.8

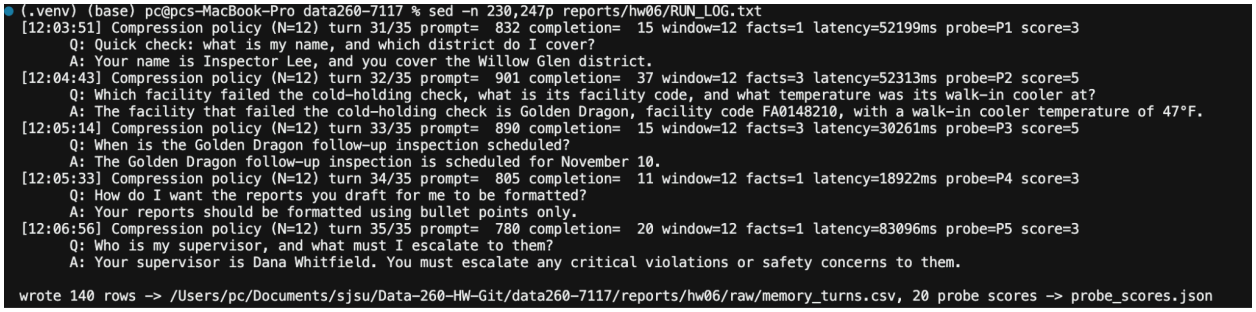


Figure 29. Compression run probes: P1 invents the name "Inspector Lee"; P5 loses what must be escalated.

The policy saves 40% of prompt tokens, but quality falls by 0.8. The 0.40 eviction threshold also removed the "user's name" fact, a weak match for the probe question, so the model guessed a name. The token saving is real; the fact cutoff is too aggressive for identity facts.

5. Memory cost estimate (10,000 × 30-turn conversations)

Pricing is $1 per 1M input tokens and $5 per 1M output tokens, applied to the measured token totals of one 30-turn run. "Incl. upkeep" adds the fact-extraction and summary calls the memory system makes every turn.

Setting	Role	Input tok/conv	Output tok/conv	Chat $/conv	Chat $ × 10k	Incl. upkeep $ × 10k
SHORT_TERM_N=4	best	18,945	1,952	0.02871	$287.05	$515.94
SHORT_TERM_N=12		28,945	2,037	0.03913	$391.30	$631.69
SHORT_TERM_N=30	worst	44,670	2,026	0.05480	$548.00	$778.03
Compression (N=12)		17,301	1,407	0.02434	$243.36	$440.85

```
        comp_rows.append({"Setting": s, "System primer": round(c.tok_system_primer.mean(), 1),
                          "Long-term summary": round(c.tok_long_term_summary.mean(), 1),
                          "Short-term window": round(c.tok_short_term_window.mean(), 1),
                          "Episodic facts": round(c.tok_episodic_facts.mean(), 1),
                          "Current message": round(c.tok_current_message.mean(), 1),
                          "Counted total": round(c.tok_counted_total.mean(), 1),
                          "Ollama prompt": round(c.prompt_tokens.mean(), 1),
                          "Mean completion": round(c.completion_tokens.mean(), 1),
                          "Model latency (ms)": round(c.model_latency_ms.mean()),
                          "Upkeep latency (ms)": round(c.aux_latency_ms.mean())})
        inp, out = c.prompt_tokens.sum(), c.completion_tokens.sum()
        ainp, aout = c.aux_prompt_tokens.sum(), c.aux_completion_tokens.sum()
        chat_cost = (inp * INPUT_PRICE_PER_M + out * OUTPUT_PRICE_PER_M) / 1e6
        total_cost = ((inp + ainp) * INPUT_PRICE_PER_M + (out + aout) * OUTPUT_PRICE_PER_M) / 1e6
        cost_rows.append({"Setting": s, "Input tok/conv": int(inp), "Output tok/conv": int(out),
                          "Upkeep in/out tok/conv": f"{int(ainp)} / {int(aout)}",
                          "Chat $/conv": f"{chat_cost:.5f}", "Chat $ x10k": f"{chat_cost * N_CONVERSATIONS:,.2f}",
                          "Incl. upkeep $ x10k": f"{total_cost * N_CONVERSATIONS:,.2f}",
                          "_total": total_cost * N_CONVERSATIONS, "_quality": p.probe_score.mean()})
        probe_rows.append({"Setting": s, **{pid: int(v) for pid, v in zip[tuple[Any, Any]](p.probe_id, p.probe_score)}})
```

```
    cost = pd.DataFrame(cost_rows)
    top_q = cost._quality.max()
    eligible = cost[cost._quality >= top_q - QUALITY_TOLERANCE]
    best = eligible.loc[eligible._total.idxmin(), "Setting"]
    worst = cost.loc[cost._total.idxmax(), "Setting"]
    cost.insert(1, "Role", ["best" if s == best else "worst" if s == worst else "" for s in cost.Setting])
```

Figure 30. Cost calculation and best/worst selection.

How best and worst were chosen. Worst is the most expensive setting: SHORT_TERM_N=30 ($548 chat-only). Best is the cheapest setting whose probe quality is within 0.5 of the top score (5.0), so a setting cannot win on price by forgetting the facts the probes test. Compression is the cheapest overall, but at 3.8 it is outside that range. That makes SHORT_TERM_N=4 the best: 48% cheaper than the worst for a 0.4-point drop in quality.

Part 4 — File-Based Memory and Consolidation

Consolidation shrank the 55-line MEMORY.md to 39 lines, merged 13 of 15 duplicate topics and resolved 3 of 5 contradictions correctly, without silently deleting anything in the test file.

1. Memory files and final AGENT.md

AGENT.md (from HW1, expanded) holds durable instructions in all five categories: project description (2), technology stack (3), coding conventions (5), testing approach (3) and prohibited actions (3). MEMORY.md holds dated facts learned from conversations, plus a `<!-- last_consolidated: ... -->` timestamp.

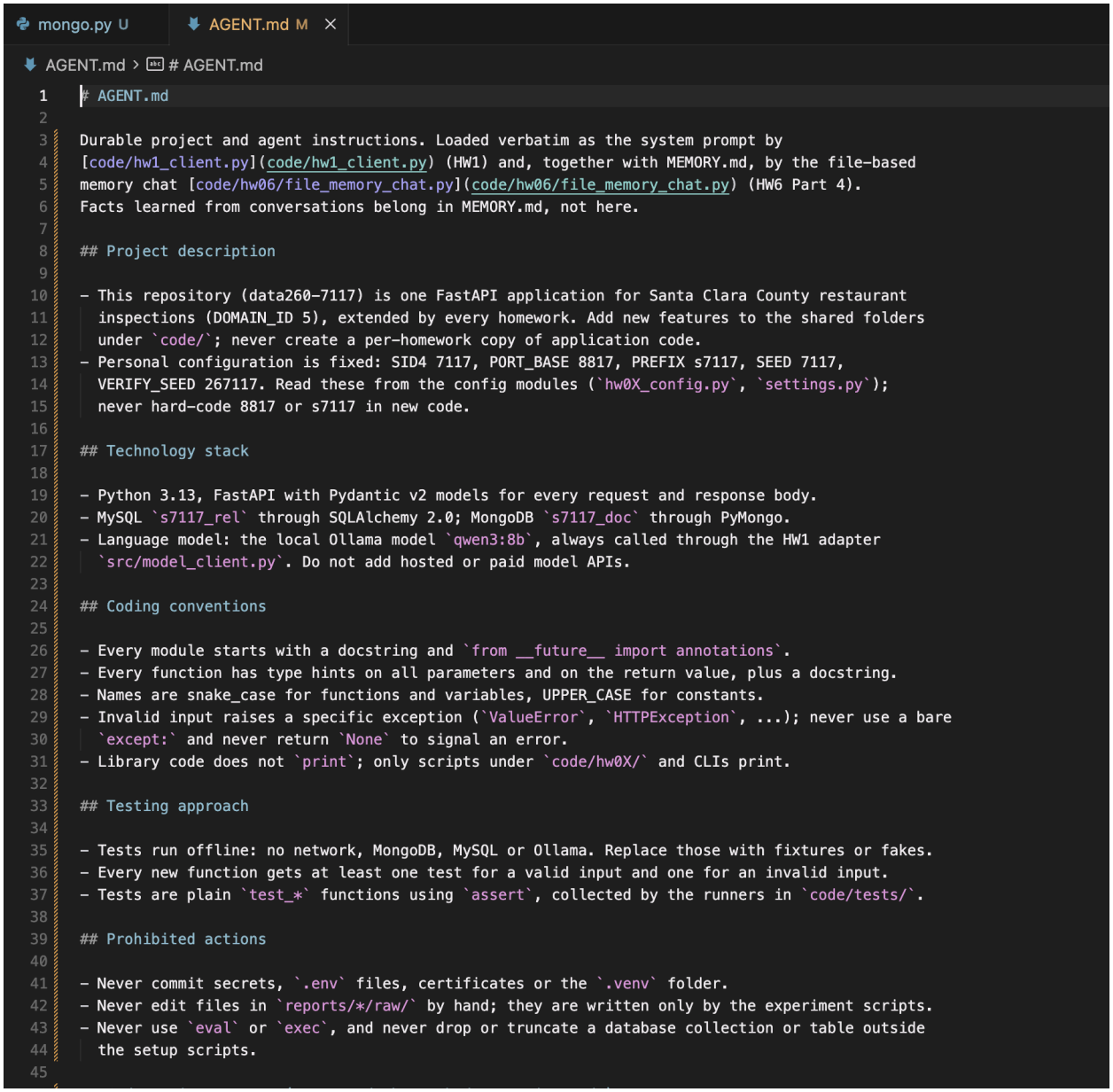


Figure 31. Final AGENT.md.

2. File-based memory functions

```python
def load_memory_files(agent_path: Path = AGENT_MD, memory_path: Path = MEMORY_MD) -> MemoryFiles:
    """Read AGENT.md and MEMORY.md (a missing MEMORY.md is created empty)."""
    if not memory_path.exists():
        memory_path.write_text(EMPTY_MEMORY)
    return MemoryFiles(agent=agent_path.read_text(), memory=memory_path.read_text(),
                       agent_path=agent_path, memory_path=memory_path)


def _strip_comments(text: str) -> str:
    return re.sub(r"<!--.*?-->", "", text, flags=re.S)


def build_system_prompt(files: MemoryFiles) -> str:
    """System prompt = AGENT.md instructions, then MEMORY.md facts (HTML comments removed)."""
    memory = _strip_comments(files.memory).strip() or "(no stored facts yet)"
    return (
        "# Agent instructions (from AGENT.md - always follow these)\n\n"
        f"{_strip_comments(files.agent).strip()}\n\n"
        "# Long-term memory (from MEMORY.md - facts learned in earlier conversations; "
        "when two facts disagree, prefer the newer date unless a fact is marked as resolved)\n\n"
        f"{memory}\n"
    )


def read_last_consolidated(memory_text: str) -> Optional[datetime]:
    """The `<!-- last_consolidated: ISO-8601 -->` timestamp, or None if missing/unparseable."""
    m = TIMESTAMP_RE.search(memory_text)
    if not m:
        return None
    try:
        ts = datetime.fromisoformat(m.group("ts").strip())
    except ValueError:
        return None
    return ts if ts.tzinfo else ts.replace(tzinfo=timezone.utc)


def count_memory_lines(memory_text: str) -> int:
    """Non-empty, non-comment lines (multi-line <!-- --> comments are skipped too)."""
    return sum(1 for line in _strip_comments(memory_text).splitlines() if line.strip())


def needs_consolidation(memory_text: str, now: Optional[datetime] = None) -> tuple[bool, list[str]]:
    """Return (required?, reasons) using the 40-line and 24-hour rules."""
    now = now or datetime.now(timezone.utc)
    reasons = []
    n = count_memory_lines(memory_text)
    if n > MAX_MEMORY_LINES:
```

Figure 32. Load both files, build the system prompt, read the timestamp, and apply the rules (> 40 non-empty non-comment lines, or > 24 h old).

3. Memory commands

```python
    def handle(self, line: str) -> Optional[str]:
        """Run a /command and return its output, or None if `line` is not a command."""
        cmd = line.strip().split()[0].lower() if line.strip().startswith("/") else ""
        return {"/memory": self.cmd_memory, "/compact": self.cmd_compact, "/dream": self.cmd_dream,
                "/help": lambda: HELP_TEXT}.get(cmd, lambda: None)()

    # ------------------------------------------------------------ commands
    def consolidation_status(self) -> str:
        text = self.memory_path.read_text()
        due, reasons = needs_consolidation(text)
        last = read_last_consolidated(text)
        return (f"MEMORY.md: {count_memory_lines(text)} non-empty non-comment lines, {len(parse_facts(text))} facts; "
                f"last consolidated: {last.isoformat() if last else 'never'}; "
                f"consolidation {'REQUIRED (' + '; '.join(reasons) + ')' if due else 'not required'}")

    def cmd_memory(self) -> str:
        files = load_memory_files(self.agent_path, self.memory_path)
        return (f"----- {self.agent_path.name} ({len(files.agent.splitlines())} lines) -----\n{files.agent.strip()}\n\n"
                f"----- {self.memory_path.name} -----\n{files.memory.strip()}\n\n{self.consolidation_status()}")

    def consolidate_memory(self) -> dict:
        before_text = self.memory_path.read_text()
        result = consolidate(before_text, planner=self.planner)
        self.memory_path.write_text(result.text)
        self.last_summary, self.last_plan_raw = result.summary(), result.plan_raw
        return self.last_summary

    def cmd_compact(self) -> str:
        s = self.consolidate_memory()
        msgs_before = len(self.history)
        if self.history:
            transcript = "\n".join(f"{m['role'].upper()}: {m['content']}" for m in self.history)
            summary = self.llm([{"role": "system", "content": COMPACT_SYSTEM},
                                {"role": "user", "content": transcript}], None).strip()
            self.history = [{"role": "user", "content": f"(Summary of the earlier conversation)\n{summary}"},
                            {"role": "assistant", "content": "Understood - continuing from that summary."}]
        out = [
            "/compact done",
            f"  MEMORY.md lines (non-empty, non-comment): {s['lines_before']} -> {s['lines_after']}",
            f"  MEMORY.md facts:                          {s['facts_before']} -> {s['facts_after']}",
            f"  conversation messages:                    {msgs_before} -> {len(self.history)}",
            f"  exact duplicates merged: {s['exact_duplicates_merged']}; contradictions resolved: "
            f"{len(s['contradictions'])}; needs review: {len(s['needs_review'])}",
        ]
        for c in s["contradictions"]:
            out.append(f"  - {c['topic']}: kept {c['kept']} over {', '.join(c['superseded'])} "
                       f"[{c['audit']['verdict']}]")
```

Figure 33. /memory, /compact, /dream, /help.

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % sed -n 1131,1148p reports/hw06/RUN_LOG.txt
===== [2026-10-16 12:41:17 PDT] make file-memory-demo-hw06 (user: rachandeep Kaur) =====
file_memory_chat.py - user: rachandeep Kaur - 2026-10-16 12:41:19 - memory: /Users/pc/Documents/sjsu/Data-260-HW-Git/data260-7117/MEMORY.md
type a message, or /help for commands
MEMORY.md: 34 non-empty non-comment lines, 25 facts; last consolidated: 2026-10-16T19:35:38+00:00; consolidation not required

rachandeep Kaur> # HW6 Part 4.3 evidence run: python code/hw06/file_memory_chat.py --script reports/hw06/file_memory_demo_script.txt

rachandeep Kaur> /help
Commands:
  /memory   show AGENT.md + MEMORY.md as loaded into the system prompt, with line count,
            last-consolidated time and whether consolidation is due
  /compact  consolidate MEMORY.md (merge duplicates, resolve contradictions) and compact the
            conversation into a summary; reports the counts before and after
  /dream    find durable long-term facts in the recent conversation and add them to MEMORY.md
  /help     this help
  /exit     quit (Ctrl-D also works)
Anything else is sent to the model with AGENT.md + MEMORY.md in the system prompt.
```

Figure 34. Start-up consolidation check and /help.

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % sed -n 1251,1270p reports/hw06/RUN_LOG.txt
rachandeep Kaur> Quick update for my notes: Bay BBQ's next routine inspection is on October 30, and I now also cover the Cambrian district
- Added "Bay BBQ's next routine inspection is on October 30" to notes.
- Added "I now also cover the Cambrian district" to notes.

rachandeep Kaur> Who is my supervisor, and when is the Golden Dragon follow-up?
- Your supervisor is Mark Chen.
- The Golden Dragon follow-up inspection is on November 10.

rachandeep Kaur> /dream
/dream added 2 fact(s) to MEMORY.md:
  - (2026-10-16) The user's supervisor is Mark Chen.
  - (2026-10-16) The Golden Dragon follow-up inspection is on November 10.

rachandeep Kaur> /compact
/compact done
  MEMORY.md lines (non-empty, non-comment): 36 -> 26
  MEMORY.md facts:                          27 -> 24
  conversation messages:                    4 -> 2
  exact duplicates merged: 0; contradictions resolved: 0; needs review: 0
```

Figure 35. /dream adds facts; /compact reports MEMORY.md lines 36 → 26, facts 27 → 24, conversation messages 4 → 2.

Problems seen in this run. /dream re-added two facts that were already stored (supervisor, follow-up date) and missed the two new ones (Bay BBQ on Oct 30, Cambrian district), because its dedup check only compares exact text. The second /compact also dropped the "Willow Glen" district fact and the "Resolved contradictions" section from MEMORY.md. That is silent loss.

4. Verifying a generated function against AGENT.md

make agent-md-check-hw06 asked qwen3:8b, with AGENT.md + MEMORY.md as the system prompt, to write parse_fact_line(line) with tests. It then checked the module against 12 AGENT.md rules and ran behaviour tests.

reports › hw06 › agent_md_check › generated_parse_fact_line.py › ...

```python
from __future__ import annotations
from datetime import date
from typing import Tuple, Optional
import re

def parse_fact_line(line: str) -> Tuple[date, str]:
    """
    Parse a line from MEMORY.md in the format '- (YYYY-MM-DD) fact text' and return a tuple
    of (datetime.date, str) with the fact text stripped. If the line is invalid, raise ValueError.
    """
    # Regular expression to match the line format
    pattern = r'^- \((\d{4}-\d{2}-\d{2})\) (.*)$'
    match = re.match(pattern, line)

    if not match:
        raise ValueError("Line does not match the required format '- (YYYY-MM-DD) fact text'")

    date_str, fact_text = match.groups()

    try:
        fact_date = date.fromisoformat(date_str)
    except ValueError:
        raise ValueError("Date is not a valid calendar date")

    return (fact_date, fact_text.strip())
```

Figure 36. The generated function.



Figure 37. AGENT.md compliance: 11/12 rules followed; behaviour: whitespace stripping fails.

The function follows 11 of the 12 rules: future import, type hints, docstring, ValueError instead of None, no print, and offline test_* tests with valid and invalid cases. It breaks one: the module has no module docstring. It also fails one behaviour test: a line with leading spaces is rejected, because the regex is anchored at `^-`.

5. Consolidation test

MEMORY.original.md has 55 lines with 15 duplicated topics and 5 planted contradictions. Consolidation merges exact duplicates in code, asks the model for a merge/resolve plan, then runs a second conflict-check pass. Every superseded value is kept under "Resolved contradictions", and each resolution is audited against the newest-wins / confirmed-wins rule.

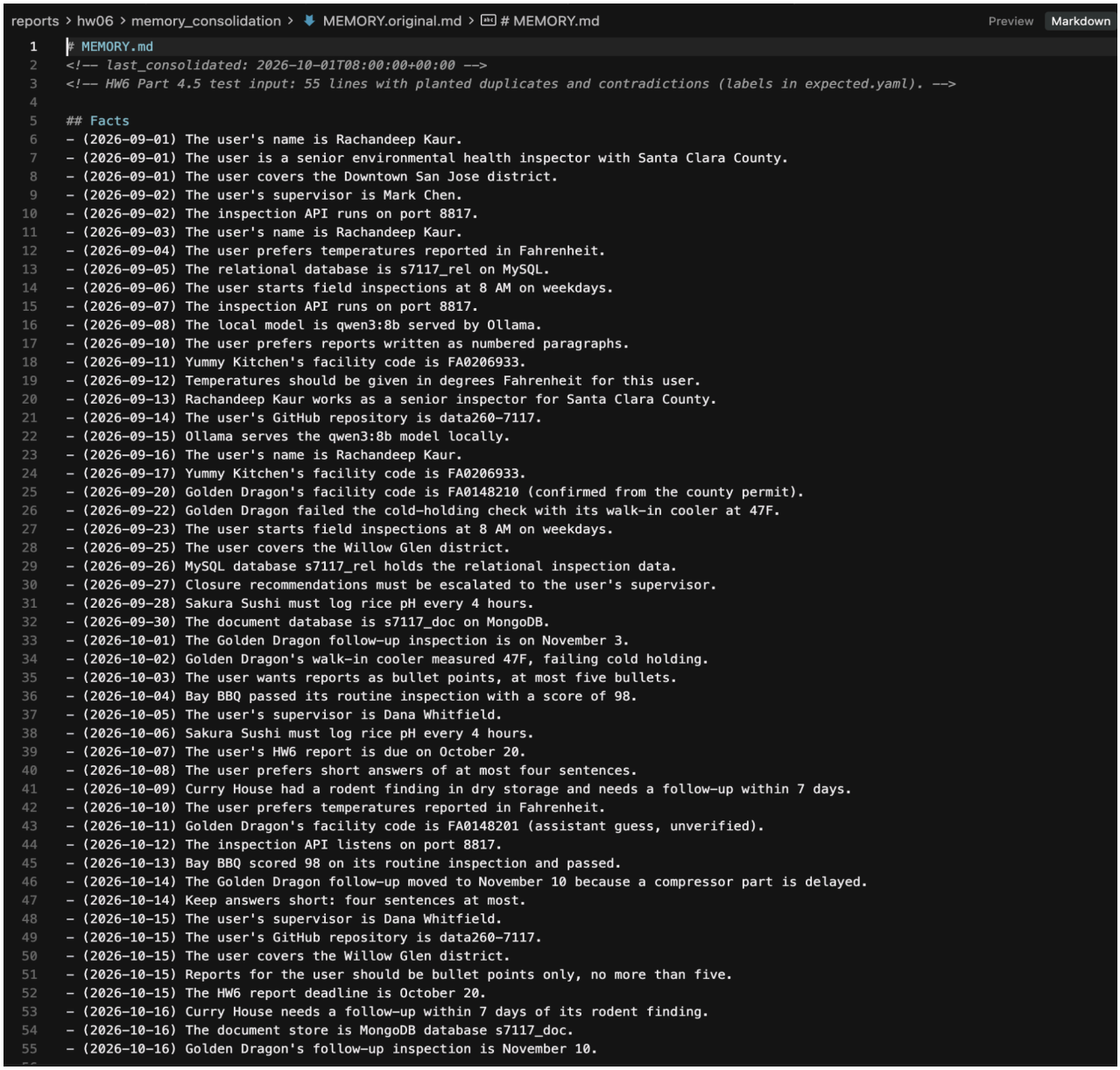


Figure 38. Original MEMORY.md

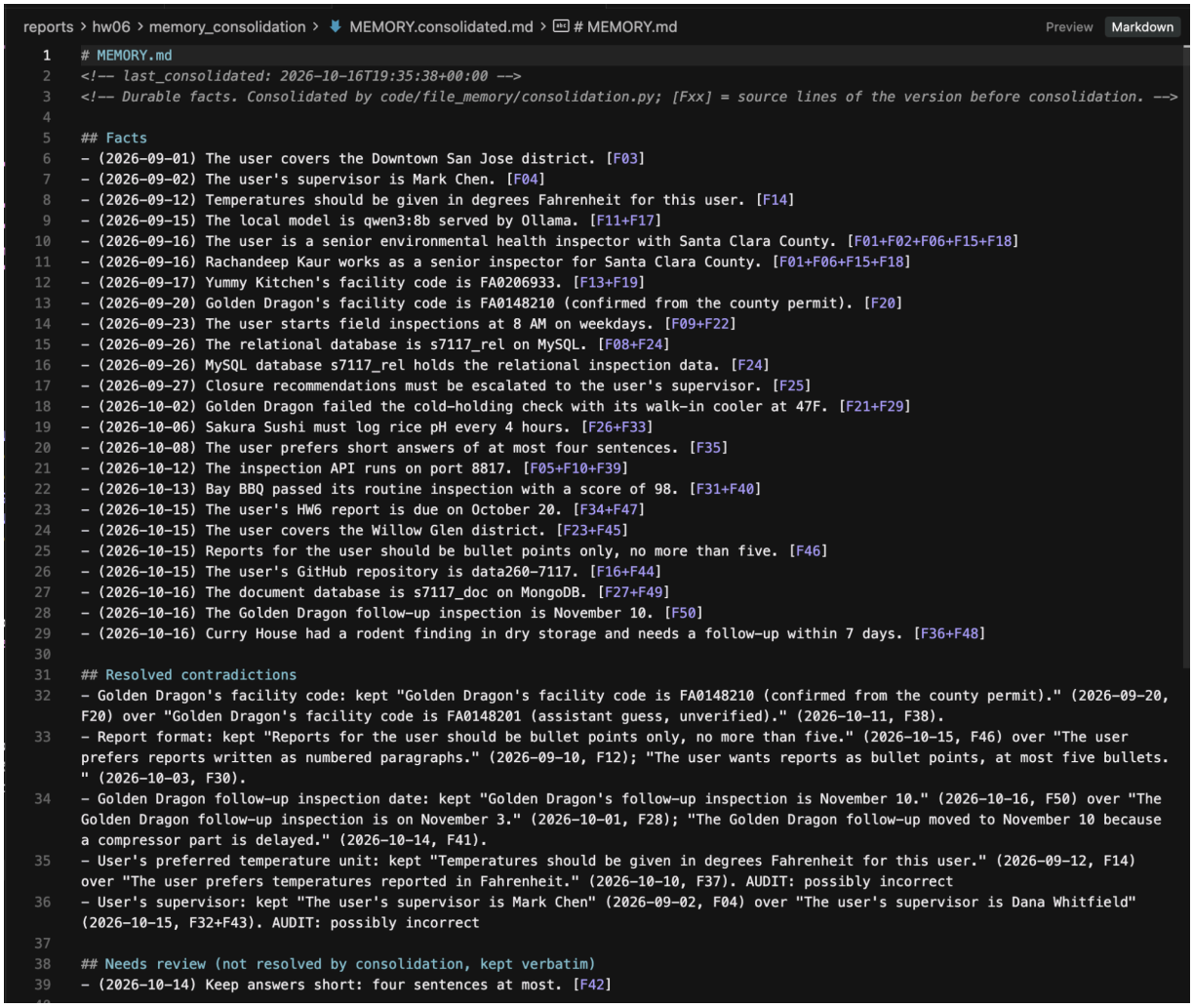


Figure 39. Consolidated MEMORY.md (39 lines).

```python
def consolidate(memory_text: str, planner: Optional[Planner] = None,
                now: Optional[datetime] = None) -> ConsolidationResult:
    planner = planner or _ollama_planner
    now = now or datetime.now(timezone.utc)
    facts = parse_facts(memory_text)
    by_id = {f["id"]: f for f in facts}

    # 2. exact duplicates: keep the newest copy as representative
    groups: dict[str, list[dict]] = {}
    for f in facts:
        groups.setdefault(_norm(f["text"]), []).append(f)
    reps, alias = [], {}
    for members in groups.values():
        rep = max(members, key=lambda f: f["date"])
        reps.append(rep)
        for m in members:
            alias[m["id"]] = rep["id"]
    exact_merged = len(facts) - len(reps)

    # 3. model plan over the de-duplicated list
    listing = "\n".join(f"{f['id']} ({f['date']}) {f['text']}" for f in sorted(reps, key=lambda f: f["id"]))
    raw = planner(CONSOLIDATE_SYSTEM, f"Facts:\n{listing}")
    try:
        plan = ConsolidationPlan.model_validate(json.loads(raw))
    except (ValueError, json.JSONDecodeError):
        plan = ConsolidationPlan()

    rep_ids = {r["id"] for r in reps}
    members_of = {rid: [f for f in facts if alias[f["id"]] == rid] for rid in rep_ids}

    # 4. validate contradictions first (they decide which ids may not appear as current facts)
    contradictions, superseded_ids = [], set[Any]()
    for c in plan.contradictions:
        kept = alias.get(c.kept)
        sup = [alias[s] for s in c.superseded if s in alias and alias[s] != kept]
        if kept not in rep_ids or not sup:
            continue
        sup = sorted(set[Any](sup) - superseded_ids)
        superseded_ids.update(sup)
        contradictions.append({"topic": c.topic, "kept": by_id[kept], "superseded": [by_id[s] for s in sup],
                               "reason": c.reason, "audit": audit_resolution(by_id[kept], [by_id[s] for s in sup])})

    out_facts, covered = [], set[Any](superseded_ids)
```

Figure 40. Exact-duplicate merge, model plan, validated contradictions, conflict-check pass and rendering with "AUDIT: possibly incorrect" flags.

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % sed -n 1029,1043p reports/hw06/RUN_LOG.txt
===== [2026-10-16 12:35:36 PDT] make consolidation-test-hw06 (user: rachandeep Kaur) =====
[12:35:38] copied MEMORY.original.md (55 lines) -> /Users/pc/Documents/sjsu/Data-260-HW-Git/data260-7117/MEMORY.md
needs_consolidation: True - ['52 non-empty, non-comment lines > 40', 'last consolidated 15 days, 11:35:38.246396 ago (> 24h)']

> /compact
/compact done
  MEMORY.md lines (non-empty, non-comment): 52 -> 34
  MEMORY.md facts:                          50 -> 25
  conversation messages:                    0 -> 0
  exact duplicates merged: 10; contradictions resolved: 5; needs review: 1
  - Golden Dragon's facility code: kept F20 over F38 [consistent_with_rule]
  - Report format: kept F46 over F12, F30 [consistent_with_rule]
  - Golden Dragon follow-up inspection date: kept F50 over F28, F41 [consistent_with_rule]
  - User's preferred temperature unit: kept F14 over F37 [possibly_incorrect]
  - User's supervisor: kept F04 over F32+F43 [possibly_incorrect]
```

Figure 41. 52 → 34 non-comment lines, 50 → 25 facts, 55 → 39 total lines.

Contradiction	Result	Superseded value kept visible	Audit flag
Golden Dragon facility code (older confirmed code must beat newer unverified guess)	correct	yes	no
Report format	correct	yes	no
Golden Dragon follow-up date (Nov 3 → Nov 10)	correct	yes	no
Supervisor (Mark Chen → Dana Whitfield)	incorrect: kept the older Mark Chen	yes	yes
District (Downtown → Willow Glen)	not detected: both kept under Facts	no	no

Incorrectly resolved contradictions.

1. Supervisor: the model kept the older "Mark Chen" over the newer "Dana Whitfield". The audit flags it as "AUDIT: possibly incorrect", and the replaced value stays visible.
2. District: neither pass noticed the conflict, so both districts are listed as current facts.
3. The model treated two Fahrenheit duplicates as a "contradiction"; the audit flagged it.

Both versions are committed under reports/hw06/memory_consolidation/, and the first-pass run is kept in raw/consolidation_run1/.

6. MongoDB memory vs. file-based memory

Criterion	MongoDB (messages / summaries / episodes)	File-based (AGENT.md + MEMORY.md)
Inspectability	Needs a client (mongosh/Compass) and queries; embeddings are unreadable; facts are spread over 3 collections.	Plain Markdown: readable, editable and diffable in Git; every change has a history.
Persistence after restart	Persists in the Docker volume and survives API restarts (the web session recalled facts from three days earlier).	Persists on disk and in Git; survives anything except a bad consolidation, which can overwrite facts (seen in /compact).
Scalability across many sessions	Scales: indexed per user/session with top-k retrieval, so the prompt grows only with k and the summary size.	Poor: the whole file goes into every prompt, one file is shared by all sessions, and it needs model-driven consolidation past 40 lines.
Facts that must not be forgotten	Risky: a fact is used only if it is retrieved (cosine ≥ threshold) or survives summarization. The compression run lost the user's name.	Better: always in the prompt, so it cannot miss retrieval. It must be protected from consolidation (e.g. pinned lines).

Best of both: keep must-not-forget instructions and identity facts in versioned files, and use MongoDB for high-volume conversation history and retrieval.

Verification (make verify-hw06)

All 14 smoke checks pass. code/hw06/verify.py checks the live system (backend on 8817, CRUD lifecycle, validation codes, POST /api/chat returns a reply plus memory, the memory/aggregate endpoints, raw data, AGENT.md and the consolidation files) and confirms code/ was not modified. It writes reports/hw06/verification.json with the homework number, SID4, commit, model/config, SEED/VERIFY_SEED, the checks and pass/fail for each.

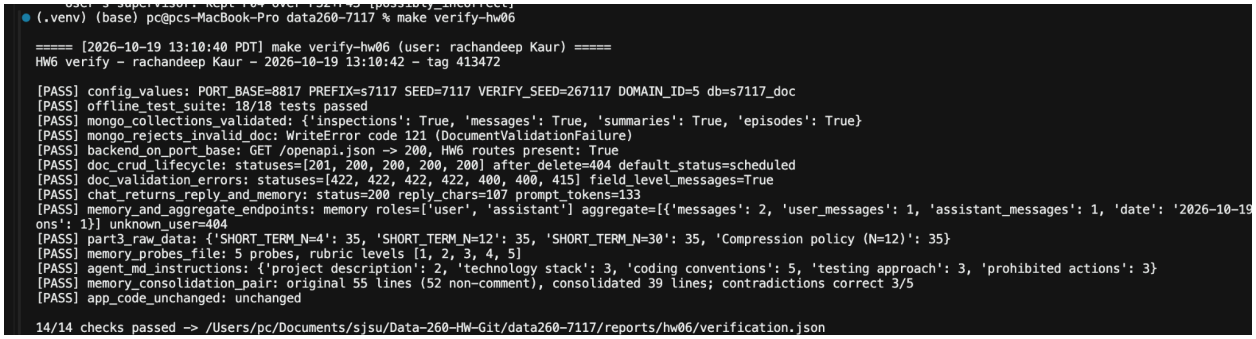


Figure 42. make verify-hw06: 14/14 checks passed.

AI Use

1. What I used an AI assistant for, and what I did myself

Claude Code (in VS Code) drafted the MongoDB document API (schemas, PyMongo connection, $jsonSchema validators, routes), the chat-memory service and endpoints, the Part 3 experiment runner and metrics script, the file-based memory package, the offline tests, verify.py, memory_probes.yaml, the 30-message conversation and the report layout. I ran MongoDB, the API, the experiments and every make target on my machine, reviewed the generated code, and checked the results against MongoDB and the raw files.

2. One AI-produced output that was wrong, and one thing I verified

Wrong: the first consolidator used a single model pass. In run 1 it left 3 of the 5 planted contradictions (supervisor, district, report format) with both versions under "Facts". The audit missed this because it only reviewed resolutions the model claimed to have made.

Verified: the Part 3 latency data. The Mac went into idle sleep during the first SHORT_TERM_N=12 run, which would have added about 6.5 hours to one turn's latency.

3. How I detected the problem / verified the result

The grader in consolidation_test.py compares the output with the ground truth in memory_consolidation/expected.yaml. It reported unresolved_both_kept for three topics, and reading the file confirmed that "Mark Chen" and "Dana Whitfield" were both current facts. The sleep was found because the run log stopped at 03:20; pmset -g log showed "Entering Sleep" at 03:20:41 and a wake at 09:49.

4. What I changed and why it works now

I added a second conflict-check pass to consolidation.py. The model re-reads the consolidated facts and names any contradictions that remain; each one is moved to "Resolved contradictions" and audited. Run 2 resolved 3 of 5 correctly, the wrong one (supervisor) is flagged instead of silent, and district is still undetected, so this is improved but not solved. I discarded the partial SHORT_TERM_N=12 run and re-ran it under caffeinate so the machine could not sleep.